

UNIDADE XIII

Curso do Meta SDK (DAT)

AUTORIA

Hugo Rodrigues Pessoni
Frederico Barbosa Relvas
Mickael Mendes Mesquita
Gustavo Novack Viana Lima



Unidade XIII - Curso do Meta SDK (DAT)

13.1 Apresentação do hardware e do SDK (DAT)

13.1.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- **Descrever** o hardware dos óculos Ray-Ban Meta (câmera, microfones, alto-falantes) e suas limitações, em especial a ausência de display.
- **Explicar** as implicações de design de um dispositivo sem tela para as soluções do hackathon (interação por visão, voz e áudio).
- **Explicar** o que é o Meta Wearables Device Access Toolkit (DAT), o que ele permite e o que ele não permite, e o que significa ele estar em developer preview.
- **Localizar** as fontes oficiais de estudo — documentação, API reference, repositório no GitHub, sample apps e fórum — e montar seu próprio roteiro de consulta.

13.1.2 Pré-requisitos

- Módulo 1 concluído, em especial 1.3 (Fundamentos de Android) e 1.4 (Permissões em runtime) — o DAT é um SDK Android e seu modelo de permissões vai parecer familiar.
- Noções de STT/TTS (tópico 1.8), úteis para entender por que a interação com os óculos é centrada em voz e áudio.
- Navegador e conta no GitHub para explorar o repositório oficial. O Android Studio só será necessário no tópico 2.3 (Setup).

13.1.3 Conceito / fundamentação

13.1.3.1 O hardware: Ray-Ban Meta

Os óculos usados em todo o programa são os **Ray-Ban Meta** — *smart glasses* com a estética clássica dos Ray-Ban (a armação *Wayfarer* é a mais conhecida), desenvolvidos pela Meta em parceria com a EssilorLuxottica. À primeira vista parecem óculos comuns; a diferença está nos sensores embutidos na armação.

FOTO: óculos Ray-Ban Meta com anotações apontando a câmera na haste esquerda, o LED de captura, os microfones e os alto-falantes nas hastes]

As características de cada componente do Ray-Ban Meta estão descritas na Tabela 18.

Tabela 18 – Características técnicas dos componentes dos óculos Ray-Ban Meta

Componente	Especificação	O que significa para você
Câmera	Ultra-wide de 12 MP	Fotos e streaming de vídeo em primeira pessoa: o app "vê" o que o usuário vê
Microfones	Array de 5 microfones	Captura de voz de qualidade mesmo em ambiente externo; entrada natural do usuário
Alto-falantes	Open-ear (abertos, direcionados ao ouvido)	Única saída de informação para o usuário; não bloqueiam o som ambiente
Display	Não possui	Nenhuma saída visual — toda resposta ao usuário é por áudio
Conectividade	Bluetooth, pareado ao smartphone via app Meta AI	Os óculos não funcionam sozinhos: dependem do telefone

Fonte: autoria própria.

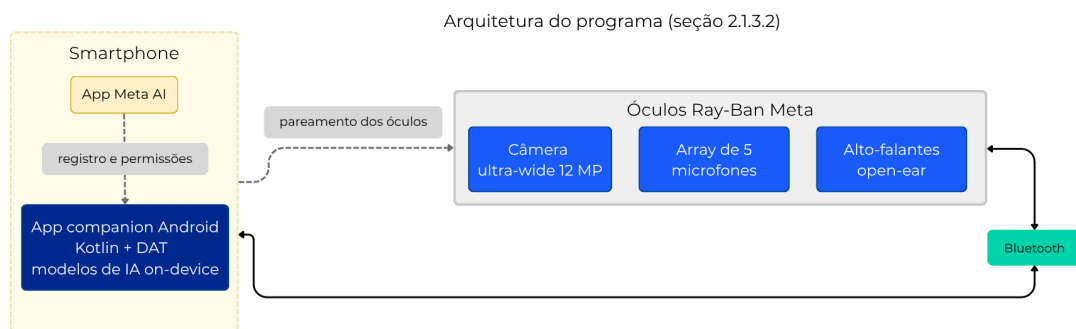
Nota 21: existem outros modelos na família de AI glasses da Meta com suporte no SDK (Ray-Ban Meta Gen 1 e Gen 2, Oakley Meta HSTN, Oakley Meta Vanguard e Meta Ray-Ban Display — este último com display). O hardware do programa é o Ray-Ban Meta **sem display**, e é para ele que você deve projetar.

13.1.3.2 Limitações que definem o jogo

Três limitações do hardware moldam qualquer solução que você vá construir:

1. **Sem display — saída só por áudio.** Não há tela, ícone, notificação visual ou realidade aumentada. Se o seu app precisa comunicar algo ao usuário, isso sai pelos alto-falantes: fala sintetizada (TTS, visto no 1.8), sons de confirmação ou música.
2. **Dependência de pareamento.** Os óculos são um periférico do smartphone: pareiam via Bluetooth através do app **Meta AI**, e é o app companion Android — rodando no telefone — que executa a lógica e os modelos de IA. Sem telefone por perto, sem app companion, não há solução.
3. **Bateria e banda limitadas.** Óculos são um dispositivo pequeno: bateria modesta e um link Bluetooth com largura de banda restrita. Isso explica decisões do SDK, como o áudio de microfone chegar ao app como mono a 8 kHz pelo perfil Bluetooth HFP (detalhes no tópico 2.6), e reforça o que você viu no 1.5 sobre consumo de energia.

Figura 17 – Arquitetura Padrão do programa



Fonte: autoria própria.

13.1.3.3 Implicações de design: pensar em visão, voz e áudio

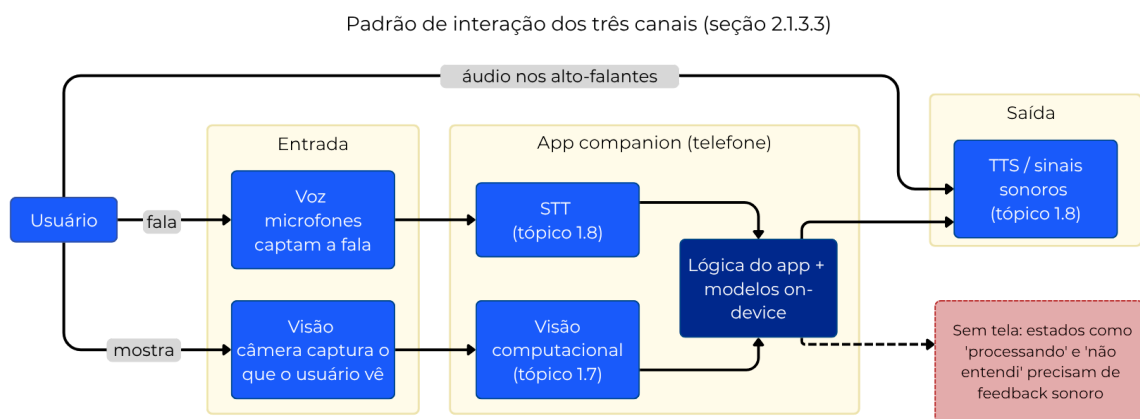
A ausência de display não é um detalhe — é a decisão de design mais importante do hackathon. Uma analogia: projetar para os Ray-Ban Meta é como projetar um assistente por telefone que enxerga. O usuário não olha para uma tela; ele **mostra** (câmera), **fala** (microfones) e **ouve** (alto-falantes).

Na prática, isso significa que toda solução do programa segue o mesmo padrão de interação (Figura 18):

- **Entrada por visão:** a câmera captura o que o usuário está vendo (uma foto ou um stream de vídeo), e a visão computacional (tópico 1.7) roda no app companion.
- **Entrada por voz:** o usuário fala; o áudio chega ao telefone e passa por STT (tópico 1.8).
- **Saída por áudio:** o resultado volta como fala (TTS) ou sinais sonoros. Respostas precisam ser curtas e objetivas — ninguém quer ouvir três parágrafos.
- **Sem confirmação visual:** estados como "estou processando" ou "não entendi" precisam de feedback sonoro explícito, senão o usuário fica falando com o vazio.

Na prática: ao avaliar sua ideia de hackathon, faça o teste dos três canais: dá para resolver com o que a câmera vê, o que o usuário fala e o que ele ouve de volta? Se a ideia exige mostrar um mapa, um gráfico ou um texto longo, ela não serve para este hardware — redesenhe a saída como áudio ou descarte.

Figura 18 – Diferentes tipos de entrada e saída para interação usuário-óculos



Fonte: autoria própria.

13.1.3.4 O SDK: Meta Wearables Device Access Toolkit (DAT)

O **Meta Wearables Device Access Toolkit (DAT)** é o SDK que permite que um app mobile (Android ou iOS; no programa, Android) acesse os sensores dos óculos. Nas palavras da documentação oficial: ele permite "conectar de forma confiável aos AI glasses da Meta e aproveitar capacidades como streaming de vídeo e captura de foto" para construir experiências hands-free dentro do seu próprio aplicativo.

O DAT foi lançado em **developer preview** — anunciado no Meta Connect de setembro de 2025. Developer preview significa, segundo o FAQ oficial: você pode baixar o SDK, construir e testar (com hardware em Developer Mode ou sem hardware, via Mock Device Kit — tópico 2.7), e compartilhar com usuários de teste por release channels; mas **não pode publicar para usuários finais** ainda.

Atenção! Por estar em preview, capacidades, APIs, versões e condições de acesso **podem mudar sem aviso**. Tudo neste módulo deve ser conferido contra a documentação oficial antes de usar — os links estão na seção de Referências. Além disso, o acesso completo ao toolkit (incluindo o Wearables Developer Center) está disponível apenas nos países com suporte oficial aos AI glasses; consulte o FAQ para a lista atual.

13.1.3.5 O que o DAT permite — e o que não permite

Veja, na Tabela 19, as funções permitidas pelo DAT.

Tabela 19 – Funcionalidades e limitações do Device Access Toolkit

✔ Permite	✗ Não permite
Streaming de vídeo da câmera dos óculos para o seu app (com seleção de resolução e frame rate)	"Hey Meta" e Meta AI: o assistente da Meta e seu wake word não fazem parte do toolkit
Captura de foto durante o stream	Rodar código <i>nos óculos</i> : seu app roda no telefone; os óculos são sensores e alto-falantes
Registro do app junto ao dispositivo do usuário e gestão de permissões (via deeplink para o app Meta AI)	Publicar para usuários finais durante o preview
Sessões com o dispositivo: o usuário pausa, retoma ou encerra tocando nos óculos, tirando-os ou fechando as hastes	Mais de uma sessão simultânea no mesmo dispositivo; e alguns recursos nativos dos óculos ficam indisponíveis durante a sessão
Testes sem hardware, com o Mock Device Kit	—

Fonte: autoria própria.

Um detalhe importante do desenho do SDK: **microfones e alto-falantes não são acessados pelo DAT**, e sim pelos perfis Bluetooth padrão do Android (HFP para captura de voz, por exemplo). O toolkit cuida de câmera, registro, permissões e sessão; o áudio passa pelo caminho Bluetooth que qualquer headset usa. Você verá isso em detalhe no tópico 2.6.

Nota 22: no Android, o SDK é distribuído como artefatos Maven (via GitHub Packages) — os principais são `mwdat-core` (registro, dispositivos, permissões, sessão), `mwdat-camera` (câmera) e `mwdat-mockdevice` (dispositivo simulado). Há também um componente de display (`MWDATDisplay`), que só se aplica ao Meta Ray-Ban Display — fora do escopo do programa. A arquitetura desses componentes é o assunto do tópico 2.2, e a instalação, do 2.3.

Nota: a versão-alvo do material é a **0.8.0** (resolve do GitHub Packages). Como o SDK está em developer preview e a versão muda com frequência, confira sempre a versão vigente no GitHub Packages e na API reference antes de fixar dependências.

13.1.4 Na prática

Este tópico não tem setup — o objetivo prático aqui é **montar seu mapa de estudo** e aprender a se orientar nas fontes oficiais. Reserve ~40 minutos e siga o roteiro.

13.1.4.1 Passo 1 — Explore o Wearables Developer Center

Abra [<https://wearables.developer.meta.com/>](https://wearables.developer.meta.com/). Este é o hub oficial: dele saem a documentação, a API reference e o cadastro de conta/organização. Repare no fluxo de 4 passos que a própria Meta propõe (conta → Developer Mode → registro de projeto → release channels) — ele é o esqueleto dos tópicos 2.3 e 2.4. Não precisa criar conta agora; isso será feito no 2.3.

13.1.4.2 Passo 2 — Faça um tour guiado pela documentação

Em [<https://wearables.developer.meta.com/docs>](https://wearables.developer.meta.com/docs), siga esta ordem de leitura (só leitura — nada de codar ainda):

1. **Setup** (`docs/develop/dat/getting-started-toolkit`) — requisitos de sistema (Android 10+, Android Studio) e como habilitar o Developer Mode no app Meta AI.
2. **Integration overview** (`docs/develop/dat/build-overview`) — a página mais importante do módulo: descreve o ciclo registro → permissões → sessão e os componentes do SDK.
3. **Session lifecycle, Permissions and registration, Microphones and speakers** — bata o olho nos títulos; são os assuntos dos tópicos 2.4 e 2.6.
4. **Mock Device Kit basics** — assunto do 2.7.

Nota 23: cada página da documentação tem um link para sua versão em Markdown puro (`.../docs-markdown/...`) — útil para colar em ferramentas de IA ou ler offline.

13.1.4.3 Passo 3 — Explore o repositório oficial no GitHub

Abra <<https://github.com/facebook/meta-wearables-dat-android>> e localize:

- **README** — visão geral e instruções de dependência Gradle (você as usará no 2.3).
- `samples/` — os sample apps oficiais, o melhor código de referência disponível durante o preview.
- `CHANGELOG.md` — em um SDK em preview, ler o changelog a cada versão não é opcional: é onde as quebras de API aparecem.
- **Discussions** — o fórum oficial da comunidade. Durante o preview, é o principal lugar onde "exemplos já existentes na internet" e soluções de problemas se concentram; procure lá antes de abrir issue.

Só para reconhecer o formato (a instalação completa é assunto do 2.3), a declaração dos artefatos no catálogo de versões (`libs.versions.toml`) e no `build.gradle.kts` — em Kotlin DSL, como visto no 1.1 — tem esta cara:

► **Código 2.1-01** — `libs.versions.toml` — trecho do README oficial (não configure ainda; ver tópico 2 — código completo no notebook companion (seção 2.1.4.3)).

► **Código 2.1-02** — `build.gradle.kts` (módulo do app) — trecho do README oficial — código completo no notebook companion (seção 2.1.4.3).

13.1.4.4 Passo 4 — Guarde os atalhos de consulta rápida

- **API reference (Android):**

<<https://wearables.developer.meta.com/docs/reference/>> — a referência é versionada; navegue sempre na versão que seu projeto usa.

- **FAQ oficial:** <<https://developers.meta.com/wearables/faq/>> — status do preview, dispositivos suportados e países.

- **Recursos para ferramentas de IA:** o repositório traz plugins/skills para Claude Code, Cursor, Copilot e afins, um endpoint `llms.txt` e um servidor MCP de documentação — vale a pena conectar sua ferramenta de IA a eles no hackathon (ver seção *AI-Assisted Development* na doc e no README).

Atenção! Erro comum: achar que o app "roda nos óculos" e sair procurando como instalar um APK neles, ou procurar no DAT uma API do "Hey Meta". Nenhum dos dois existe: seu código **roda no app companion Android**, os óculos entram como câmera/microfone/alto-falante, e o acionamento da sua solução é responsabilidade do seu app — não do assistente da Meta.

Na prática: monte hoje um documento de time com esses links fixados (docs, API reference na versão certa, repo, Discussions, FAQ). No dia do hackathon, os minutos gastos procurando "onde estava aquela página" fazem falta — e, num SDK em preview, a página de ontem pode ter mudado.

13.1.5 Resumo / cheatsheet

- Ray-Ban Meta: câmera ultra-wide 12 MP + array de 5 microfones + alto-falantes open-ear; **sem display** — toda saída é áudio.
- Os óculos dependem de pareamento Bluetooth com o smartphone (via app Meta AI); a lógica e a IA rodam no app companion Android.
- Design das soluções: entrada por visão (câmera) e voz (mics), saída por áudio (TTS/sons); sempre dar feedback sonoro de estado.

- DAT = SDK que dá ao seu app mobile acesso aos sensores dos óculos: streaming de vídeo, captura de foto, registro, permissões e sessões.
- Microfones e alto-falantes não passam pelo DAT: usam os perfis Bluetooth padrão do Android (detalhes no 2.6).
- "Hey Meta" e Meta AI **não fazem parte** do toolkit; e seu código não roda nos óculos.
- Developer preview: pode construir, testar (glasses em Developer Mode ou Mock Device Kit) e distribuir a testadores via release channels — mas não publicar; tudo pode mudar, confira sempre a doc oficial.
- Mapa de estudo: Developer Center (hub) → docs (guias) → API reference (versionada) → GitHub `facebook/meta-wearables-dat-android` (README, `samples/`, CHANGELOG, Discussions) → FAQ.

13.1.6 Referências

- META WEARABLES DEVELOPER CENTER / META. Disponível em: <<https://wearables.developer.meta.com/>>. Hub oficial: criação de conta/organização e ponto de partida para docs e API reference.
- DOCUMENTAÇÃO DO DEVICE ACCESS TOOLKIT / META. Disponível em: <<https://wearables.developer.meta.com/docs>>. Guias de setup, integração, sessões, permissões, áudio e Mock Device Kit.
- SETUP — DEVICE ACCESS TOOLKIT / META. Disponível em: <<https://wearables.developer.meta.com/docs/develop/dat/getting-started-toolkits/>>. Requisitos de sistema, dispositivos suportados e habilitação do Developer Mode.
- INTEGRATION OVERVIEW — DEVICE ACCESS TOOLKIT / META. Disponível em: <<https://wearables.developer.meta.com/docs/develop/dat/build-overview/>>. Ciclo registro → permissões → sessão e os componentes do SDK (core, camera, display).

- API REFERENCE — WEARABLES / META. Disponível em: <https://wearables.developer.meta.com/docs/reference/>. Referência versionada das APIs Android e iOS do toolkit.
- META WEARABLES DEVICE ACCESS TOOLKIT FOR ANDROID / FACEBOOK (GITHUB). Disponível em: <https://github.com/facebook/meta-wearables-dat-android>. SDK via GitHub Packages, sample apps, CHANGELOG e fórum de Discussions.
- FREQUENTLY ASKED QUESTIONS — WEARABLES / META. Disponível em: <https://developers.meta.com/wearables/faq/>. Significado do developer preview, dispositivos suportados e disponibilidade por país.

13.1.6.1 Para ir além



INTRODUCING THE META WEARABLES DEVICE ACCESS TOOLKIT / META. Disponível em:

<https://developers.meta.com/blog/introducing-meta-wearables-device-access-toolkit/>. Post de lançamento, com a visão da Meta para casos de uso do toolkit.



AI-ASSISTED DEVELOPMENT — DEVICE ACCESS TOOLKIT / META. Disponível em:

<https://wearables.developer.meta.com/docs/develop/dat/ai-assisted/>. Como conectar Claude Code, Cursor, Copilot e servidores MCP à documentação do DAT.

13.2 Arquitetura da solução

13.2.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- **Explicar** o modelo de arquitetura óculos $\rightleftharpoons$ Bluetooth $\rightleftharpoons$ app companion e o papel de cada componente.
- **Justificar** por que os modelos de IA e o agente rodam no celular, e não nos óculos.

- **Identificar** por onde os dados entram (câmera, microfones) e saem (alto-falantes) da solução, e quais limites o Bluetooth impõe a esse tráfego.
- **Relacionar** cada checkpoint do hackathon — entrada por câmera/mic, saída por áudio e IA local — ao componente da arquitetura que o satisfaz.

13.2.2 Pré-requisitos

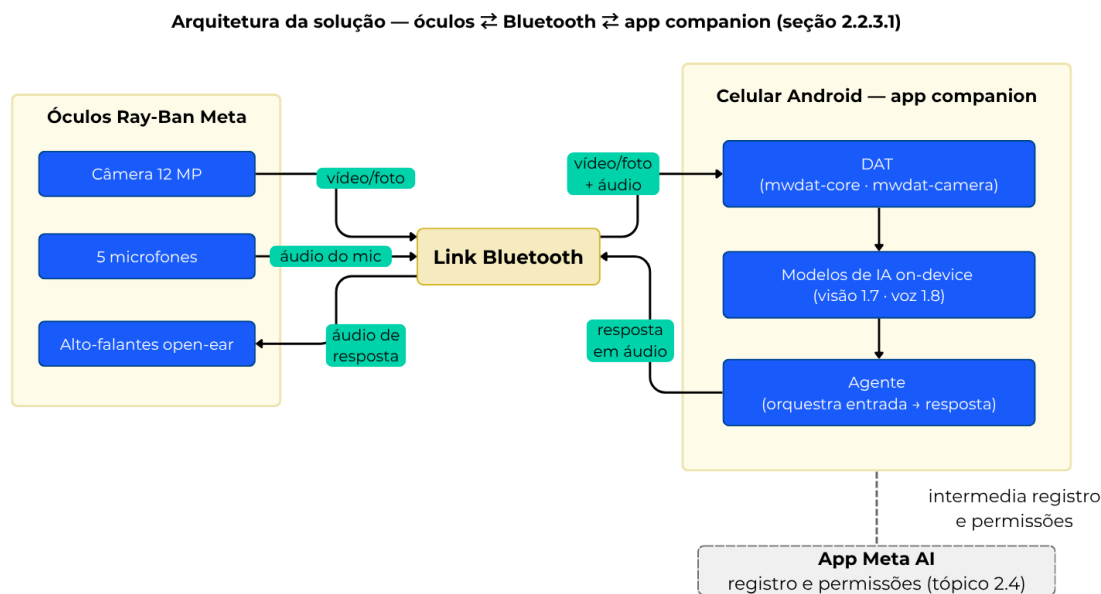
- **Tópico 2.1** — Hardware dos óculos e visão geral do Wearables Device Access Toolkit (DAT).
- **Tópico 1.6** — Conceito de edge AI (execução de modelos on-device).
- **Ferramentas:** nenhuma nova. Este é um tópico conceitual; o setup do ambiente vem no tópico 2.3.

13.2.3 Conceito / fundamentação

13.2.3.1 A arquitetura em uma frase

Os óculos capturam e reproduzem; o celular pensa. Toda solução do hackathon segue o mesmo modelo: os óculos Ray-Ban Meta fornecem os sensores (câmera ultra-wide de 12 MP, array de 5 microfones) e os atuadores (alto-falantes open-ear); o Bluetooth transporta esses dados; e um ***app companion*** — no celular/notebook; no nosso programa, um app Android — recebe os dados, executa os modelos de IA e o agente, e devolve a resposta em áudio (Figura 19).

Figura 19 – Arquitetura da solução: óculos, Bluetooth® e app companion



Fonte: autoria própria.

13.2.3.2 Os óculos são um periférico de I/O

Os Ray-Ban Meta não têm display, têm bateria pequena e um chip dimensionado para captura e conectividade — não para rodar redes neurais. Mais importante: **o DAT não executa código de terceiros nos óculos**. Não existe "instalar seu app nos óculos"; toda integração é um app mobile que acessa os sensores do dispositivo remotamente.

A analogia útil: pense nos óculos como um **fone Bluetooth com câmera embutida**. Assim como você não instala aplicativos no seu fone — ele só envia o áudio do microfone e toca o que o celular manda —, os óculos só enviam vídeo/áudio e reproduzem a resposta. I/O de entrada e saída, nada mais.

13.2.3.3 O elo: Bluetooth

A conexão entre óculos e celular usa Bluetooth, e isso define os limites físicos da solução:

- **Vídeo:** o streaming da câmera trafega por Bluetooth Classic, cuja banda é limitada. Por isso o DAT trabalha com resoluções e frame rates modestos e adapta a qualidade automaticamente quando a banda aperta (detalhes no tópico 2.5).

- **Áudio do microfone:** chega ao app pelo perfil HFP (Hands-Free Profile), o mesmo usado por fones em chamadas — mono e com taxa de amostragem baixa (8 kHz dado da doc oficial, pode mudar com versões). Detalhes no tópico 2.6.
- **Áudio de saída:** o app toca áudio pelo canal Bluetooth padrão do sistema, como faria com qualquer fone; os alto-falantes open-ear dos óculos reproduzem (tópico 2.6).

Atenção! Bluetooth não é Wi-Fi. Projetar pensando em vídeo full-HD a 60 fps é o erro conceitual mais comum deste módulo — o link não entrega isso, e o DAT vai degradar a qualidade sozinho. Assuma frames comprimidos, em resolução moderada, e desenhe sua solução em torno disso.

13.2.3.4 O app companion: onde tudo roda

O app Android que sua equipe vai construir concentra três responsabilidades:

1. **Falar com os óculos** — via DAT (`mwdat-core`, `mwdat-camera`), que gerencia descoberta do dispositivo, sessão e streams. O app não abre sockets Bluetooth manualmente: o registro e as permissões são intermediados pelo **app Meta AI** instalado no celular do usuário (fluxo detalhado no tópico 2.4).
2. **Executar a IA** — os modelos de visão (tópico 1.7) e de fala (tópico 1.8) rodam on-device no celular, com as técnicas de edge AI do tópico 1.6.
3. **Orquestrar o agente** — a lógica que decide o que fazer com cada entrada: transcrever a fala, analisar o frame, montar a resposta e enviá-la aos alto-falantes.

13.2.3.5 Por que a compute fica no celular

Três razões se somam:

- **Física:** rodar um modelo de visão exige processamento, memória e energia que não cabem em uma armação de óculos sem esquentar, pesar e drenar a bateria em minutos. O celular tem SoC com aceleradores, gigabytes de RAM e bateria ordens de grandeza maior.

- **Plataforma:** mesmo que coubesse, o DAT não oferece execução de código nos óculos — a única superfície programável da solução é o app mobile.
- **Requisito do programa:** o checkpoint de **IA local** exige que a inferência aconteça on-device, sem depender de nuvem. O celular é o único lugar da arquitetura capaz de satisfazer isso.

Na prática: essa divisão define onde sua equipe gasta esforço no hackathon. Os óculos são iguais para todos os times — o que diferencia sua solução é o que roda no celular: a escolha do modelo, a otimização da inferência e a inteligência do agente.

13.2.3.6 Arquitetura × checkpoints

Os *checkpoints* de acordo com a arquitetura estão descritos na Tabela 20, a seguir.

Tabela 20 – Relação entre checkpoints, componentes da arquitetura e caminhos dos dados

Checkpoint	Componente que satisfaz	Caminho do dado
Entrada por câmera	Câmera dos óculos + stream do DAT	Óculos → Bluetooth → app companion
Entrada por microfone	Microfones dos óculos + áudio HFP	Óculos → Bluetooth → app companion
Saída por áudio	Alto-falantes open-ear + áudio BT do sistema	App companion → Bluetooth → óculos
IA local	Modelos on-device no app companion	Inferência inteira no celular, sem nuvem

Fonte: autoria própria.

13.2.4 Na prática

Este tópico é conceitual, então o exercício aqui é **rastrear um ciclo completo de dados** pela arquitetura — o mesmo ciclo que sua solução do hackathon vai executar milhares de vezes:

1. O usuário fala ou aponta o olhar para algo → **microfones/câmera dos óculos** capturam.

2. Os dados atravessam o **Bluetooth** e chegam ao app companion.
3. O app roda **STT** sobre o áudio (tópico 1.8) e/ou o **modelo de visão** sobre o frame (tópico 1.7) — tudo on-device.
4. O **agente** decide a resposta e gera a fala com **TTS** (tópico 1.8).
5. O áudio volta pelo **Bluetooth** e toca nos **alto-falantes** dos óculos.

Em código, o esqueleto do app companion espelha exatamente esses blocos. Não se preocupe com os detalhes de cada chamada agora — cada uma tem seu próprio tópico:

► **Código 2.2-01** – Esqueleto conceitual do app companion — código completo no notebook companion (seção 2.2.4).

Repare no que o código **não** contém: nenhuma linha roda nos óculos, nenhum socket Bluetooth é aberto à mão, nenhuma chamada vai para a nuvem. O DAT abstrai o transporte; seu trabalho é o que acontece dentro do `collect`.

Atenção! Erro comum de primeira semana: procurar os óculos no `adb devices` ou perguntar "como faço deploy nos óculos?". O alvo do deploy é **sempre o celular**. Os óculos nunca aparecem para você como um dispositivo Android — apenas como um `Device` dentro do DAT.

Na prática: como o ciclo inteiro (captura → BT → inferência → BT → áudio) soma latências, meça o tempo de ponta a ponta desde o início do desenvolvimento. Uma resposta que demora 5 segundos mata a experiência de voz — e otimizar a inferência no celular (tópico 1.6) é onde você tem mais controle sobre esse tempo.

13.2.5 Resumo / cheatsheet

- Arquitetura: óculos (câmera/mic/alto-falantes) ⇔ Bluetooth ⇔ app companion Android (DAT + modelos de IA + agente).
- Os óculos são **periférico de I/O**: capturam e reproduzem; não executam código de terceiros.
- Toda a compute — modelos e agente — roda **no celular**, on-device.

- O Bluetooth é o gargalo: vídeo comprimido em resolução/fps modestos; áudio de mic via HFP mono.
- O app **não** fala Bluetooth direto com os óculos: o DAT e o app Meta AI intermediam registro, permissões e sessão.
- Checkpoints: câmera e mic = entrada pelos óculos; áudio = saída pelos óculos; IA local = inferência no celular, sem nuvem.
- Deploy é sempre no celular; os óculos aparecem para o app apenas como um dispositivo do DAT.

13.2.6 Referências

- INTEGRATION OVERVIEW — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/build-overview/>. Visão geral oficial da integração: sessões, componentes do toolkit (core e câmera) e uso de microfones/alto-falantes via Bluetooth.
- WEARABLES DEVICE ACCESS TOOLKIT — DOCUMENTAÇÃO. META. Disponível em: <https://wearables.developer.meta.com/docs>. Portal central da documentação do DAT, incluindo setup, integração Android e API reference.
- META-WEARABLES-DAT-ANDROID. META (GITHUB). Disponível em: <https://github.com/facebook/meta-wearables-dat-android>. Repositório oficial do SDK Android, com apps de exemplo que implementam a arquitetura deste tópico.
- FAQ — META WEARABLES. Disponível em: <https://developers.meta.com/wearables/faq/>. Perguntas frequentes sobre o programa de desenvolvedores e o toolkit.

2.2.6.1 Para ir além



MICROPHONES AND SPEAKERS — META WEARABLES DEVELOPER CENTER. Disponível em:

<https://wearables.developer.meta.com/docs/microphones-and-speakers/>.
Detalha o caminho de áudio (HFP e saída) que o tópico 2.6 vai aprofundar.



SESSION LIFECYCLE — META WEARABLES DEVELOPER
CENTER. Disponível em:

<https://wearables.developer.meta.com/docs/lifecycle-events/>. Ciclo de
vida da sessão entre app e óculos, base para o tópico 2.4.

13.3 Setup do ambiente

13.3.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar como o Meta Wearables Device Access Toolkit (DAT) é distribuído via GitHub Packages e por que o Gradle precisa de autenticação para baixá-lo.
- Criar e configurar um personal access token (classic) do GitHub com o escopo `read:packages`, guardando-o de forma segura no projeto.
- Configurar o projeto Android para o DAT: repositório Maven no `settings.gradle.kts`, artefatos no `libs.versions.toml` e dependências no `build.gradle.kts`.
- Configurar o `AndroidManifest.xml` com as permissões exigidas, os metadados `APPLICATION_ID` e `CLIENT_TOKEN` (atestação) e o intent filter com URI scheme.
- Verificar os requisitos mínimos de ambiente (Android, Android Studio, óculos e app Meta AI) antes de começar a desenvolver.

13.3.2 Pré-requisitos

- Tópico 2.1 (Hardware e SDK) e 2.2 (Arquitetura) — você já conhece o papel do app companion e do DAT na arquitetura óculos ⇔ Bluetooth ⇔ Android.

- Tópico 1.3 (Fundamentos de Android) — projeto Android com Gradle (Kotlin DSL) e estrutura de módulos.
- Android Studio Flamingo ou mais recente instalado, e um dispositivo/emulador com Android 10 (API 29) ou superior. **Verificar:** requisitos de versão na página oficial de Setup, pois o DAT está em developer preview e eles podem mudar: <https://wearables.developer.meta.com/docs/getting-started-toolkit/>
- Uma conta no GitHub (necessária para gerar o token de acesso aos pacotes).

13.3.3 Conceito / fundamentação

13.3.3.1 O que "setup" significa aqui

Diferente de uma biblioteca comum, o DAT envolve três partes que precisam estar alinhadas antes da primeira linha de código útil: o **seu projeto Android** (que baixa e referencia o SDK), o **GitHub Packages** (de onde o SDK é baixado, com autenticação) e o **app Meta AI** no celular (que faz a ponte com os óculos e valida a identidade do seu app) (Figura 20). Este tópico cobre a configuração do projeto; o registro do app junto ao Meta AI e a criação de sessão são assunto do tópico 2.4.

Figura 20 – As três partes que o setup precisa alinhar



Fonte: autoria própria.

13.3.3.2 Requisitos mínimos do ambiente

Segundo a documentação oficial (na data de escrita):

- **Android:** 10 (API 29) ou superior — o mesmo requisito do app Meta AI.
- **IDE:** Android Studio Flamingo ou mais recente.

- **Óculos suportados:** Ray-Ban Meta (Gen 1 e Gen 2) e Meta Ray-Ban Display. Também é possível testar sem hardware usando o Mock Device Kit (tópico 2.7).
- **App Meta AI:** versão v254 ou superior, com developer mode habilitado.
- **Firmware dos óculos:** v20+ (Ray-Ban Meta) ou v21+ (Meta Ray-Ban Display).

Verificar: todos esses números mudam com frequência em developer preview. Confira a página de Setup e a matriz "Version Dependencies" antes do hackathon: <https://wearables.developer.meta.com/docs/version-dependencies/>

Nota 24: para habilitar o developer mode, abra o app Meta AI, vá em Settings > App Info e toque 5 vezes no número da versão do app — o toggle "Developer Mode" aparece. Sem ele, seu app em desenvolvimento não consegue conversar com os óculos.

13.3.3.3 Distribuição via GitHub Packages

O DAT não está no Maven Central nem no Google Maven: ele é publicado no **GitHub Packages**, o registro de pacotes do GitHub, dentro do repositório oficial `facebook/meta-wearables-dat-android`. Para o Gradle, isso é só mais um repositório Maven — com uma diferença importante: **o GitHub Packages exige autenticação para baixar pacotes, mesmo os públicos**. É como um armário de materiais do laboratório: qualquer aluno pode pegar, mas precisa passar o crachá na porta.

Esse "crachá" é um **personal access token (classic)** — um PAT — da sua conta GitHub, com pelo menos o escopo `read:packages` (permissão de leitura de pacotes, nada mais). O token entra como senha nas credenciais do repositório Maven; o username não é necessário.

Atenção! O token é uma credencial pessoal. Ele nunca deve ir para o repositório Git do time — por isso a configuração usa `local.properties` (que já fica no `.gitignore` dos projetos Android) ou uma variável de ambiente.

13.3.3.4 Identidade do app: APPLICATION_ID, CLIENT_TOKEN e atestação

Quando seu app pede acesso aos óculos, o app Meta AI precisa confiar nele. Esse processo de verificação de autenticidade chama-se **atestação** (attestation) e

usa dois valores declarados como `<meta-data>` no manifest: `APPLICATION_ID` e `CLIENT_TOKEN`. Ambos são obtidos ao registrar seu app no Wearables Developer Center (seção "Manage projects").

A assinatura do app (App Signature) não é obrigatória para a atestação, mas o Meta AI a usa para verificar a autenticidade; identificadores incorretos fazem a conexão falhar com erro.

Nota 25: a atestação **não é usada em Developer Mode** — apps em modo de desenvolvimento rodam com lógica local, sem canal de release. Nesse caso, você pode omitir os valores ou usar `0`. Para o hackathon, é assim que seu time vai trabalhar.

13.3.3.5 O caminho de volta: intent filter com URI scheme

Fluxos como registro e permissões acontecem **dentro do app Meta AI**: seu app envia o usuário para lá e, ao final, o Meta AI precisa devolvê-lo ao seu app. Ele faz isso abrindo uma URI com um **scheme customizado** (ex.: `myexampleapp://...`). Para que o Android saiba qual Activity abrir, você declara um **intent filter** com `action VIEW`, categorias `DEFAULT` e `BROWSABLE`, e o seu scheme. Sem esse intent filter, o usuário fica "preso" no app Meta AI e o fluxo nunca conclui.

Na prática: no hackathon, cada time terá seu próprio app companion. Escolha um scheme único e específico do seu app (ex.: `meuappoculos`) — schemes genéricos podem colidir com outros apps instalados no mesmo celular de teste, e aí o callback abre o app errado.

13.3.4 Na prática

Passo a passo para deixar o Gradle sincronizando com o DAT em um projeto Android existente (Kotlin DSL). O resultado deste tópico é o **sync verde**; inicializar o SDK e registrar o app são o tópico 2.4.

13.3.4.1 Passo 1 — Crie o personal access token no GitHub

1. No GitHub, acesse Settings > Developer settings > Personal access tokens > **Tokens (classic)** > Generate new token (classic).
2. Marque **apenas** o escopo `read:packages`.
3. Copie o token gerado (formato `ghp_...`) — ele só é exibido uma vez.

Verificar: o caminho exato da UI do GitHub muda com o tempo; instruções oficiais em <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

Atenção! Tokens *fine-grained* nem sempre funcionam com GitHub Packages — a documentação do DAT pede explicitamente um token classic.

13.3.4.2 Passo 2 — Guarde o token fora do controle de versão

Duas opções (escolha uma):

```
# local.properties (raiz do projeto — já ignorado pelo Git)

github_token=ghp_seu_token_aqui
```

Código 2.3-01 · também no notebook companion, seção 2.3.4.2

Ou, via variável de ambiente no terminal:

```
export GITHUB_TOKEN=ghp_seu_token_aqui
./gradlew installDebug # rodando a partir da raiz do
projeto
```

Código 2.3-02 · também no notebook companion, seção 2.3.4.2

Atenção! Erro comum: colocar o token no `gradle.properties` e commitá-lo. O GitHub detecta tokens vazados em pushes e os **revoga automaticamente**, e o build de todo o time quebra. Use `local.properties` ou variável de ambiente, sempre.

13.3.4.3 Passo 3 — Adicione o repositório Maven no settings.gradle.kts

► **Código 2.3-03** – settings.gradle.kts — código completo no notebook companion (seção 2.3.4.3).

Nota 26: o repositório vai no settings.gradle.kts (dentro de dependencyResolutionManagement), não no build.gradle.kts do módulo — esse é o padrão moderno de resolução centralizada de dependências.

13.3.4.4 Passo 4 — Declare os artefatos no libs.versions.toml

► **Código 2.3-04** – gradle/libs.versions.toml — código completo no notebook companion (seção 2.3.4.4).

Nota 27: o material usa **mwdat 0.8.0**. Como o SDK é versionado, confira versões mais recentes em https://github.com/orgs/facebook/packages?repo_name=meta-wearables-dat-android

Os três artefatos: `mwdat-core` (núcleo do SDK), `mwdat-camera` (streaming e captura — tópico 2.5) e `mwdat-mockdevice` (dispositivo simulado — tópico 2.7).

13.3.4.5 Passo 5 — Adicione as dependências no build.gradle.kts do app

► **Código 2.3-05** – app/build.gradle.kts — código completo no notebook companion (seção 2.3.4.5).

13.3.4.6 Passo 6 — Configure o AndroidManifest.xml

► **Código 2.3-06** – Permissões usadas pelo DAT: comunicação Bluetooth com os óculos + rede — código completo no notebook companion (seção 2.3.4.6).

Os `§{...}` são *manifest placeholders*, preenchidos pelo Gradle:

► **Código 2.3-07** – app/build.gradle.kts — código completo no notebook companion (seção 2.3.4.6).

Nota 28: como visto no 1.4, `BLUETOOTH_CONNECT` é permissão de runtime (API 31+): declarar no manifest não basta, o usuário precisa conceder em tempo de execução. O fluxo completo de permissões do DAT será visto no tópico 2.4.

Na prática: sem `BLUETOOTH_CONNECT` concedida, o app companion não fala com os óculos — nem câmera, nem microfone, nem áudio. Esse é o primeiro item a checar quando "nada funciona" no dia do hackathon.

13.3.4.7 Passo 7 — Sync e validação

No Android Studio: **File > Sync Project with Gradle Files**. Se o sync concluir sem erros e os artefatos `mwdat-*` aparecerem em External Libraries, o setup está pronto. O passo a passo completo depende de conta GitHub e das versões atuais dos pacotes.

Erro comum: o sync falha com `401 Unauthorized` (ou `Could not GET 'https://maven.pkg.github.com/...'`). Causas típicas, na ordem em que valem a checagem: token ausente (esqueceu o `local.properties` ou a variável de ambiente), token sem o escopo `read:packages`, token expirado/revogado, ou nome da chave errado (`github_token` no arquivo vs. o que o `settings.gradle.kts` lê). Corrigido o token, rode o sync de novo — o Gradle não tenta sozinho.

13.3.5 Resumo / cheatsheet

- Requisitos: Android 10+, Android Studio Flamingo+, óculos Ray-Ban Meta (Gen 1/2) ou Meta Ray-Ban Display, app Meta AI v254+ com developer mode — ou Mock Device Kit (2.7).
- O DAT é distribuído via GitHub Packages: repositório Maven `https://maven.pkg.github.com/facebook/meta-wearables-dat-android`.
- GitHub Packages exige autenticação mesmo para pacotes públicos: PAT **classic** com escopo `read:packages`, usado como `password` (username vazio).

- Token fica em `local.properties` (`github_token=...`) ou na variável de ambiente `GITHUB_TOKEN` — nunca no Git.
- Repositório vai no `settings.gradle.kts` (`dependencyResolutionManagement`); versões e artefatos (`mwdat-core`, `mwdat-camera`, `mwdat-mockdevice`) no `libs.versions.toml`; `implementation(...)` no `build.gradle.kts`.
- Manifest: permissões `BLUETOOTH`, `BLUETOOTH_CONNECT`, `INTERNET` (+ `CAMERA` para mock device com câmera do celular).
- `APPLICATION_ID` e `CLIENT_TOKEN` (meta-data) servem à atestação; em Developer Mode podem ser `0`.
- Intent filter com URI scheme próprio permite ao app Meta AI devolver o usuário ao seu app.

13.3.6 Referências

- SETUP — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/getting-started-toolkit/> — requisitos de plataforma, óculos suportados, versões mínimas e habilitação do developer mode.
- INTEGRATE WEARABLES DEVICE ACCESS TOOLKIT INTO YOUR ANDROID APP — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/build-integration-android/> — passo a passo oficial: manifest, Gradle, token e dependências (fonte dos trechos de configuração deste tópico).
- META-WEARABLES-DAT-ANDROID — REPOSITÓRIO OFICIAL NO GITHUB. Disponível em: <https://github.com/facebook/meta-wearables-dat-android> — código-fonte, packages publicados e sample app completo para comparação.

- MANAGING YOUR PERSONAL ACCESS TOKENS — GITHUB DOCS.

Disponível em:

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens> — criação e gerenciamento do token classic com escopos.

13.3.6.1 Para ir além



MANAGE PROJECTS — META WEARABLES DEVELOPER CENTER. Disponível em:

<https://wearables.developer.meta.com/docs/manage-projects> — onde registrar seu app e obter `APPLICATION_ID` e `CLIENT_TOKEN` para builds fora do Developer Mode.



VERSION DEPENDENCIES — META WEARABLES DEVELOPER CENTER. Disponível em:

<https://wearables.developer.meta.com/docs/version-dependencies/> — matriz de compatibilidade entre SDK, app Meta AI e firmware dos óculos.

13.4 Registro e ciclo de sessão

13.4.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar por que o SDK precisa ser inicializado uma única vez por processo e o que acontece quando uma API é chamada antes da inicialização.
- Implementar o fluxo de registro do app com `Wearables.startRegistration` e o de desregistro com `Wearables.startUnregistration`, observando `registrationState` e `devices`.
- Descrever o ciclo de vida de uma `Session` (IDLE → STARTING → STARTED ⇌ PAUSED → STOPPING → STOPPED) e o papel do `Stream` como capability anexada à sessão.

- Implementar a reação correta a pause/resume e ao encerramento da sessão, sem tentar "reviver" uma sessão encerrada.
- Configurar o monitoramento de disponibilidade do dispositivo via `devicesMetadata` (campo `linkState`).

13.4.2 Pré-requisitos

- **Tópico 1.2 — Coroutines e Flow:** todo o estado do toolkit é exposto como `StateFlow/SharedFlow`; você vai coletar flows o tempo todo.
- **Tópico 2.2 — Arquitetura:** entender o papel do app Meta AI como intermediário entre seu app e os óculos.
- **Tópico 2.3 — Setup:** projeto Android com as dependências `mwdat` (versão 0.8.0) configuradas, manifest com o scheme de callback e Developer Mode habilitado no app Meta AI.
- Óculos Ray-Ban Meta pareados com o app Meta AI — ou o Mock Device Kit, que será visto no tópico 2.7.

13.4.3 Conceito / fundamentação

13.4.3.1 Onde estamos na arquitetura

Seu app nunca fala diretamente com os óculos. Como visto no 2.2, o caminho é: **seu app** → **SDK (DAT)** → **app Meta AI** → **Bluetooth** → **óculos**. Isso explica duas características centrais deste tópico: o *registro* acontece dentro do app Meta AI (é ele quem pergunta ao usuário se confia no seu app), e o *estado da sessão* é comandado pelo sistema — seu app **observa** o estado, não o controla sozinho.

Uma analogia: o app Meta AI é a portaria de um prédio. O registro é o seu cadastro na portaria (feito uma vez); a sessão é cada visita — a portaria pode suspender ou encerrar a visita a qualquer momento (óculos guardados, outro visitante prioritário), e cabe a você reagir, não discutir.

13.4.3.2 Inicialização do SDK: uma vez por processo

Antes de qualquer outra chamada, o SDK precisa ser inicializado:

```
Wearables.initialize(context)
```

Código 2.4-01 · também no notebook companion, seção 2.4.3.2

A regra é: **uma vez por processo, no startup do app**. O lugar natural é o `onCreate()` da sua classe `Application`, garantindo que a inicialização aconteça antes de qualquer `Activity` ou `ViewModel` tocar no SDK.

Atenção! O erro típico é chamar qualquer API do toolkit antes de `initialize`. Segundo a documentação oficial, isso resulta em `WearablesError.NOT_INITIALIZED` — e os métodos do objeto `Wearables` lançam `WearablesException` quando o SDK não foi inicializado. O sintoma clássico: o app funciona quando você abre pela tela principal, mas quebra quando o Android recria o processo direto em outra tela. Inicializar na `Application` elimina essa classe de bug.

Nota 29: `initialize` retorna um `DatResult<Unit, WearablesError>` — o tipo de resultado padrão do toolkit, no estilo do `Result` do Kotlin (com `fold`, `getOrElse`, `onSuccess`/`onFailure`).

13.4.3.3 Registro: apresentando seu app aos óculos

O **registro** estabelece a confiança entre seu app e a plataforma dos óculos. Ele acontece **uma única vez**, através do app Meta AI, com os óculos conectados. Depois de registrado, seu app aparece na lista de apps conectados do Meta AI, e o usuário pode desregistrá-lo por lá a qualquer momento.

O SDK expõe duas chamadas e dois pontos de observação:

- `Wearables.startRegistration(activity)` — abre o app Meta AI, onde o usuário completa o fluxo de registro. Quando ele termina, o resultado é processado automaticamente e refletido no estado.
- `Wearables.startUnregistration(activity)` — mesmo mecanismo, para desregistrar.
- `Wearables.registrationState:`
`StateFlow<RegistrationState>` — o estado atual do registro.

- `Wearables.devices: StateFlow<Set<DeviceIdentifier>>` — o conjunto de dispositivos que o SDK pode usar.
- `RegistrationState` é um **enum** com cinco constantes (Tabela 21). O erro de registro **não** fica no estado — vem de um fluxo à parte, `Wearables.registrationErrorStream: Flow<RegistrationError>`

Tabela 21 – Estados do `RegistrationState` e seus respectivos significados

Estado	Significado
UNAVAILABLE	Registro indisponível no momento (restrição de sistema — ex.: app Meta AI ausente).
AVAILABLE	Registro pode ser iniciado.
REGISTERING	Fluxo de registro em andamento.
REGISTERED	App registrado com sucesso.
UNREGISTERING	Fluxo de desregistro em andamento.

Fonte: autoria própria.

Atenção! Em Developer Mode, **apenas um app de terceiros pode permanecer registrado por vez**. Registrar um novo app desregistra automaticamente o anterior. No hackathon, se um colega instalar e registrar o build dele no mesmo celular, o seu app perde o registro silenciosamente — se `registrationState` voltar para `Available` sem você pedir, é isso.

Nota 30: como visto no 2.3, o retorno do Meta AI para o seu app depende do intent filter com o URI scheme declarado no manifest. Sem ele, o fluxo de registro não consegue voltar.

13.4.3.4 Sessão: acesso sustentado ao dispositivo

Com o app registrado, o acesso aos sensores dos óculos acontece dentro de uma **device session** — um período de acesso sustentado às capacidades do dispositivo (em contraste com *transactions*, interações curtas controladas pelo sistema, como o "Hey Meta").

O fluxo é:

1. `Wearables.createSession(deviceSelector)` cria a sessão para um dispositivo que case com o seletor (`AutoDeviceSelector` deixa o SDK

escolher; `SpecificDeviceSelector` usa um dispositivo que você indicar). A resolução do dispositivo é imediata, contra o snapshot atual de `Wearables.devices`.

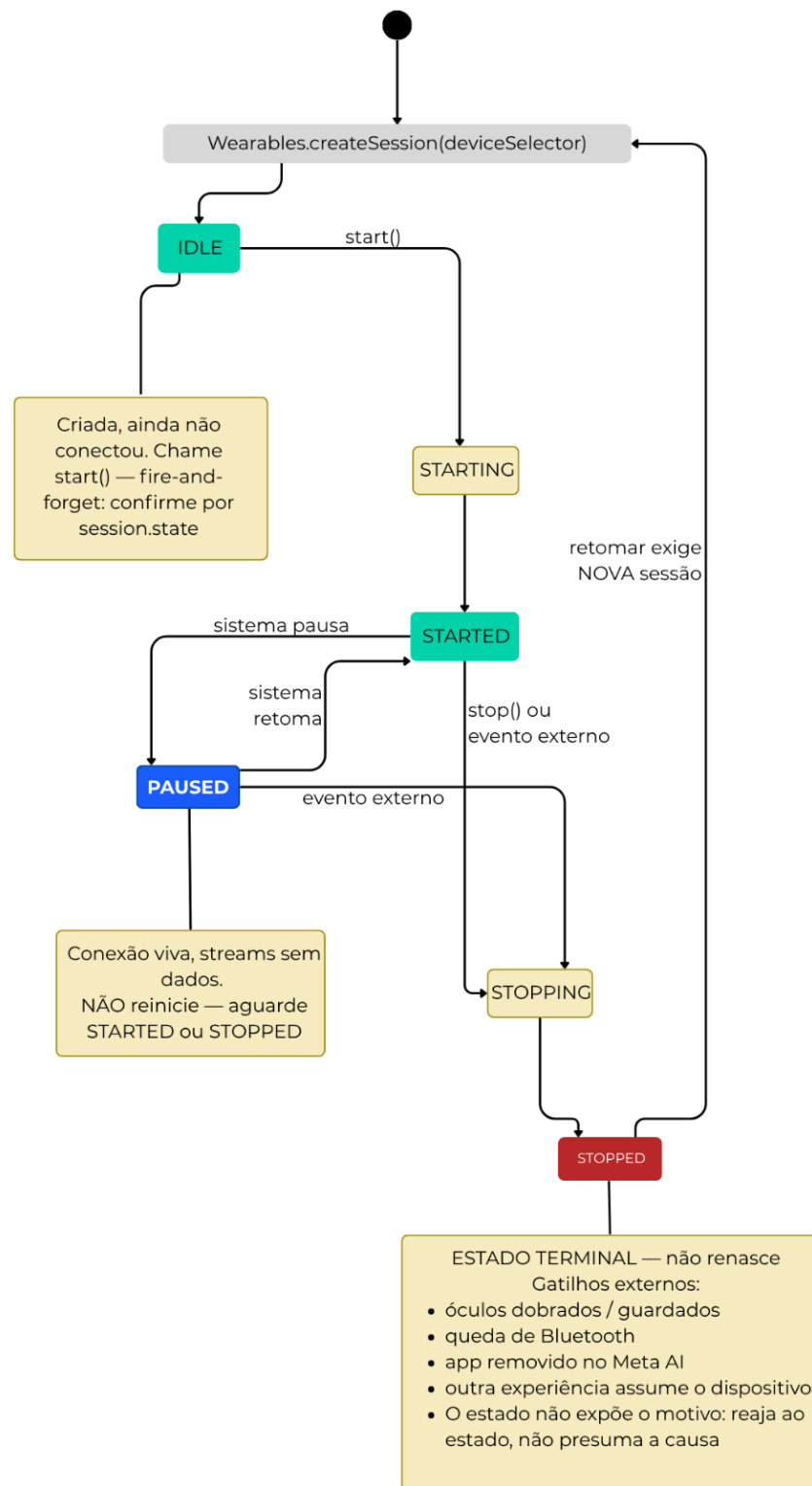
2. A sessão nasce em `IDLE`. Chame `session.start()` para conectar.
3. **Capabilities** são anexadas à sessão — o `Stream` de câmera entra via `session.addStream(...)` (detalhes de câmera no tópico 2.5; microfone e áudio no 2.6). Quando a sessão para, todas as capabilities anexadas param juntas (*cascading stop*).

`createSession` pode falhar com `DeviceSessionError.NO_ELIGIBLE_DEVICE` (...) ou `DeviceSessionError.SESSION_ALREADY_EXISTS` — existe garantia de **uma sessão ativa por dispositivo** (singleton). Se já há uma sessão não encerrada, crie capabilities nela em vez de criar outra sessão.

13.4.3.5 O ciclo de estados da sessão

O estado da sessão que você criou é exposto em `session.state: StateFlow<DeviceSessionState>` (Figura 21):

Figura 21 – Ciclo de vida da DeviceSession



Fonte: autoria própria.

Diferentes tipos de estados de sessão são descritos na Tabela 22.

Tabela 22 – Diferentes tipos de estados de sessão

Estado	Significado	O que seu app faz
IDLE	Criada, ainda não conectou.	Chamar <code>start()</code> .
STARTING	Conectando ao dispositivo.	Aguardar; mostrar progresso.
STARTED	Ativa; capabilities funcionam.	Trabalhar (adicionar stream etc.).
PAUSED	Suspensa temporariamente pelo sistema.	Segurar o trabalho e aguardar.
STOPPING	Encerrando.	Aguardar.
STOPPED	Encerrada — estado terminal .	Liberar recursos; nova sessão se preciso.

Fonte: autoria própria.

Três regras importam mais que tudo:

- `start()` e `stop()` são **fire-and-forget**: retornam na hora e o trabalho acontece em background. A confirmação vem observando `session.state`. Chamar `start()` fora de `IDLE`, ou `stop()` em sessão já parada, é no-op.
- `STOPPED` é **terminal**. Sessão parada não renasce: crie outra com `createSession`.
- Erros chegam por um flow separado, `session.errors`:
`SharedFlow<DeviceSessionError>` (ex.: `DEVICE_DISCONNECTED`, `DEVICE_POWERED_OFF`).

Observe `state` e `errors` juntos.

O sistema pode mudar o estado sem seu app pedir: um gesto do usuário abre outra experiência, outro app inicia uma sessão, o usuário dobra ou tira os óculos (desconectando o Bluetooth), remove seu app no Meta AI, ou a conectividade entre o Meta AI e os óculos cai. O estado **não expõe o motivo** da transição — reaja ao estado observado sem presumir a causa.

13.4.3.6 Pause e resume

Quando a sessão vai para `PAUSED`:

- A conexão com o dispositivo continua viva.
- Os streams **param de entregar dados** enquanto durar a pausa.
- A retomada é do sistema: a sessão volta a `STARTED` (ou vai a `STOPPED`) sozinha.

Atenção! Não tente reiniciar uma sessão enquanto ela está pausada. O padrão correto é: em `PAUSED`, segure o processamento e aguarde; em `STARTED`, retome; em `STOPPED`, libere recursos e ofereça ao usuário a opção de reconectar.

13.4.3.7 Disponibilidade do dispositivo

Antes de criar sessão — e para reagir a desconexões — monitore os metadados do dispositivo:

- `Wearables.devicesMetadata[deviceId]` expõe um `StateFlow` com metadados, incluindo o campo `linkState` (`LinkState`: `CONNECTED` / `CONNECTING` / `DISCONNECTED`).
- `Wearables.getDeviceState(deviceId)` expõe um `StateFlow<DeviceState>` com a saúde do dispositivo (ex.: nível térmico). O estado da sessão vem de `session.state` (visto acima) — não existe um método de estado de sessão por dispositivo no DAT 0.8.

Nota 31: o estado da sessão é único — `session.state`: `StateFlow<DeviceSessionState>` (`IDLE`/`STARTING`/`STARTED`/`PAUSED`/`STOPPING`/`STOPPED`). O DAT 0.8 não tem um "estado de sessão por dispositivo"; a saúde do dispositivo (térmica) vem de `Wearables.getDeviceState(deviceId)`: `StateFlow<DeviceState>`.

O caso físico mais comum: **dobrar as hastes** dos óculos desconecta o Bluetooth, para os streams ativos e força a sessão para `STOPPED`. Reabrir as hastes

restaura o Bluetooth quando os óculos estão por perto, **mas não recria a sessão** — seu app deve criar uma nova quando o dispositivo voltar a ficar disponível.

Na prática: no hackathon, isso acontece na demo: alguém guarda os óculos na capa entre um teste e outro, e o app "para de funcionar". Um app bem-feito observa `devicesMetadata` e, quando `available` volta a `true`, mostra um botão de reconexão (ou reconecta sozinho). É a diferença entre uma demo que trava e uma que se recupera na frente da banca.

13.4.4 Na prática

Vamos montar o esqueleto de registro + sessão de um app companion. Os trechos assumem as dependências e o manifest do tópico 2.3 e usam `viewModelScope` (tópico 1.2).

13.4.4.1 Passo 1 — Inicializar o SDK na Application

► **Código 2.4-02** — `GlassesApp.kt` — registre esta classe no `AndroidManifest` (`android:name=".GlassesA` — código completo no notebook companion (seção 2.4.4.1).

Erro comum: inicializar dentro de uma `Activity`. Se o processo for recriado em outra tela (ou um `WorkManager` acordar o app), alguma chamada ao SDK acontece antes do `initialize` e você recebe `WearablesException/NOT_INITIALIZED`. Na `Application`, a ordem é garantida.

13.4.4.2 Passo 2 — Observar registro e dispositivos

► **Código 2.4-03** — `RegistrationViewModel.kt` — código completo no notebook companion (seção 2.4.4.2).

Como `registrationState` é um `StateFlow`, você recebe o valor atual imediatamente ao coletar — a UI já nasce no estado certo.

13.4.4.3 Passo 3 — Disparar registro e desregistro

► **Código 2.4-04** – Chamados a partir de cliques na UI; precisam de uma Activity, — código completo no notebook companion (seção 2.4.4.3).

Repare que não há callback nem valor de retorno: o resultado do fluxo é processado automaticamente pelo SDK e aparece no `registrationState` que você já observa no Passo 2. É o padrão *dispare e observe o estado*, que se repete na sessão.

13.4.4.4 Passo 4 — Criar a sessão e observar estado + erros

► **Código 2.4-05** – `SessionViewModel.kt` — código completo no notebook companion (seção 2.4.4.4).

Erro comum: tratar `start()` como operação síncrona e chamar `addStream` na linha seguinte. Como a sessão ainda está em `STARTING`, você recebe erro — o stream só pode ser adicionado depois de `STARTED` (adicionar com a sessão em `IDLE` retorna `DeviceSessionError.SESSION_IDLE`). A solução é reagir à transição para `STARTED` dentro do collect, como acima.

13.4.4.5 Passo 5 — Anexar um Stream à sessão

► **Código 2.4-06** – Dentro de `onSessionLive()`, com a sessão em `STARTED` — código completo no notebook companion (seção 2.4.4.5).

Só um stream por sessão: uma segunda chamada retorna `DeviceSessionError.CAPABILITY_ALREADY_ADDED` (remova antes com `removeStream()`). Configuração de qualidade, frame rate e o processamento dos frames são assunto do tópico 2.5.

13.4.4.6 Passo 6 — Monitorar a disponibilidade do dispositivo

► **Código 2.4-07** – Visão por dispositivo: útil para decidir QUANDO criar/recriar a sessão — código completo no notebook companion (seção 2.4.4.6).

Na prática: sem óculos físicos na equipe, todo esse fluxo — registro, estados da sessão, pause e desconexão — pode ser exercitado com o Mock Device Kit, que inclusive simula transições de estado. É o assunto do tópico 2.7; o código acima permanece o mesmo, o que muda é o dispositivo por trás.

13.4.5 Resumo / cheatsheet

- `Wearables.initialize(context)`: uma vez por processo, na `Application`; API chamada antes disso → `NOT_INITIALIZED/WearablesException`.
- Registro é feito no app Meta AI: `startRegistration(activity)/startUnregistration(activity)`; resultado chega via `Wearables.registrationState` (`UNAVAILABLE/AVAILABLE/REGISTERING/REGISTERED/UNREGISTERING`); erros via `Wearables.registrationErrorStream`.
- Dispositivos visíveis: `Wearables.devices` (`StateFlow<Set<DeviceIdentifier>>`); metadados e disponibilidade: `Wearables.devicesMetadata[id]` (campo `linkState`).
- `createSession(selector)` → sessão nasce `IDLE`; erros: `NO_ELIGIBLE_DEVICE`, `SESSION_ALREADY_EXISTS` (uma sessão ativa por dispositivo).
- Ciclo: `IDLE` → `STARTING` → `STARTED` ⇌ `PAUSED` → `STOPPING` → `STOPPED`; `STOPPED` é terminal — retomar = criar nova sessão.
- `start()/stop()` são fire-and-forget: confirme pelo `session.state`; erros vêm por `session.errors`.
- Em `PAUSED`: conexão viva, streams sem dados; não reinicie — aguarde `STARTED` ou `STOPPED`.
- Dobrar as hastes: Bluetooth cai, streams param, sessão → `STOPPED`; reabrir restaura o Bluetooth, mas a sessão nova é por sua conta.

13.4.6 Referências

- INTEGRATE WEARABLES DEVICE ACCESS TOOLKIT INTO YOUR ANDROID APP / META. Disponível em:
<https://wearables.developer.meta.com/docs/build-integration-android/>.
Passo a passo oficial de manifest, Gradle, inicialização, registro, sessão e stream no Android.
- SESSION LIFECYCLE / META. Disponível em:
<https://wearables.developer.meta.com/docs/lifecycle-events/>. Estados da sessão, transições comuns, pause/resume e monitoramento de disponibilidade do dispositivo.
- PERMISSIONS AND REGISTRATION / META. Disponível em:
<https://wearables.developer.meta.com/docs/permissions-requests/>. Modelo de registro via app Meta AI, comportamento multi-dispositivo e regras do Developer Mode.
- WEARABLES OBJECT — API REFERENCE (ANDROID DAT 0.8) / META. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_core_wearables.
Assinaturas de initialize, startRegistration, startUnregistration, createSession, registrationState, devices e getDeviceState.
- SESSION CLASS — API REFERENCE (ANDROID DAT 0.8) / META. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_core_session_session. Ciclo DeviceSessionState, semântica fire-and-forget de start/stop, errors e addStream/removeStream.
- META-WEARABLES-DAT-ANDROID / META (GITHUB). Disponível em:
<https://github.com/facebook/meta-wearables-dat-android>. Repositório oficial com o sample app Android completo para comparar com sua implementação.

13.4.6.1 Para ir além



MOCK DEVICE KIT BASICS / META. Disponível em: <https://wearables.developer.meta.com/docs/mock-device-kit/>. Como simular dispositivo e transições de sessão sem hardware (base para o tópico 2.7).



SETUP / META. Disponível em: <https://wearables.developer.meta.com/docs/getting-started-toolkit/>. Requisitos de versão do app Meta AI, firmware dos óculos e Developer Mode.

13.5 Câmera dos óculos

13.5.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar a relação entre `DeviceSession` e `Stream` no Meta Wearables Device Access Toolkit (DAT) e o papel de cada um no acesso à câmera.
- Implementar a abertura de um stream de vídeo dos óculos no app companion, coletando frames e estados via Flow.
- Implementar a captura de foto durante um streaming ativo e tratar os formatos de retorno (`PhotoData.HEIC` e `PhotoData.Bitmap`).
- Configurar resolução (`VideoQuality`) e frame rate no `StreamConfiguration`, escolhendo valores adequados ao caso de uso.
- Comparar os trade-offs de qualidade impostos pela banda do Bluetooth Classic e antecipar o comportamento do rebaixamento automático de qualidade.

13.5.2 Pré-requisitos

- Tópico 2.3 (Setup): projeto Android com os artefatos do DAT configurados no Gradle (`mwdat-core`, `mwdat-camera`) e manifest preparado.

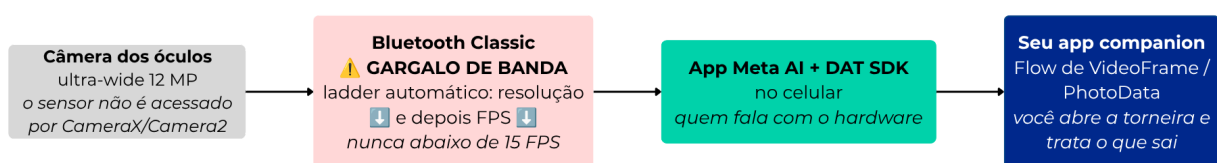
- Tópico 2.4 (Registro e sessão): app registrado no Meta AI, permissão `Permission.CAMERA` do wearable concedida e conceitos de estado de sessão.
- Tópico 1.2 (Coroutines e Flow): `Flow`, `StateFlow` e `collect` são usados em todo o código deste tópico.
- Android Studio com o SDK do DAT versão 0.8.0 (`Verificar`: o toolkit está em developer preview e a versão muda com frequência; confira a mais recente em GitHub Packages do repositório oficial).

13.5.3 Conceito / fundamentação

13.5.3.1 Como o vídeo chega ao seu app

A câmera ultra-wide dos óculos não é acessada como uma câmera do Android (nada de CameraX/Camera2 aqui). Quem fala com o hardware é o app Meta AI, e o seu app recebe os dados por meio do DAT: os óculos capturam o vídeo, enviam pelo Bluetooth Classic ao celular e o toolkit entrega os frames ao seu código como um `Flow`. Pense no DAT como um "encanamento" já pronto: você não controla o sensor diretamente — você abre uma torneira (o stream) e decide o que fazer com o que sai dela (Figura 22).

Figura 22 – Como o vídeo dos óculos chega ao seu app



Fonte: autoria própria.

13.5.3.2 Session e Stream: quem faz o quê

O acesso à câmera é organizado em duas camadas:

- `DeviceSession`: representa o acesso do seu app a um dispositivo. Você a cria com `Wearables.createSession(...)`, passando um seletor de dispositivo — `AutoDeviceSelector` (o SDK escolhe pelo usuário) ou

`SpecificDeviceSelector` (quando seu app oferece UI de seleção). A criação e os estados da sessão foram vistos no tópico 2.4.

- `Stream`: é a *capability* de mídia anexada à sessão, criada com `session.addStream(config)`. É ele quem entrega vídeo (`videoStream`), estado (`state`), erros (`errorStream`) e captura de foto (`capturePhoto()`).

Dois detalhes importantes dessa hierarquia:

1. `addStream()` apenas **registra** a *capability*; é o `stream.start()` que ativa a câmera nos óculos e começa a entrega de frames.
2. O stop é **em cascata**: quando a sessão-pai para, todos os streams dela param automaticamente. Você também pode parar um stream individualmente com `stream.stop()` (ou removê-lo com `Session.removeStream()`). Após `stop()`, o stream é invalidado e não pode ser reutilizado — crie outro se precisar.

Nota 32: `Stream` implementa `Closeable`, e `close()` delega para `stop()`. Isso permite usar o stream com `use { ... }` do Kotlin quando fizer sentido.

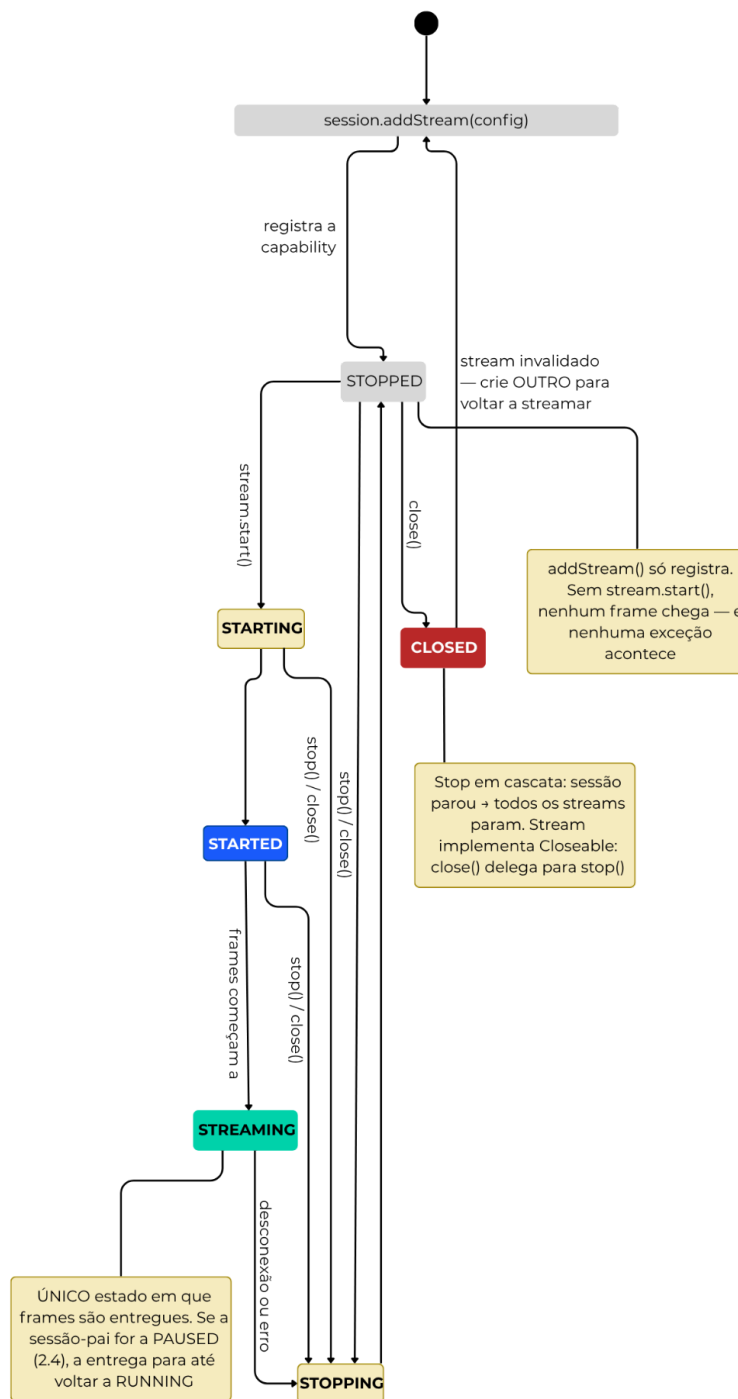
13.5.3.3 O ciclo de vida do stream

O estado do stream é exposto em `stream.state`: `StateFlow<StreamState>` e transita assim:

- `STOPPED` → `STARTING` → `STARTED` → `STREAMING` — quando o dispositivo conecta e o streaming começa;
- `STREAMING` → `STOPPING` → `STOPPED` — em desconexão ou erro;
- qualquer estado → `STOPPING` → `STOPPED` → `CLOSED` — em `stop/close`.

Os frames de vídeo **só são entregues no estado** `STREAMING`. Se a sessão for pausada pelo sistema (estado `PAUSED`, visto no 2.4 — por exemplo, quando outro app assume o dispositivo), os streams param de entregar dados até a sessão voltar a `RUNNING` (Figura 23).

Figura 23 – Ciclo de vida do Stream



Fonte: autoria própria.

Atenção! Fechar as hastes dos óculos derruba o Bluetooth, para os streams ativos e leva a sessão a **STOPPED**. Reabrir as hastes restaura o Bluetooth, mas **não** reinicia a sessão — seu app precisa criar uma nova quando o dispositivo voltar a ficar disponível.

13.5.3.4 Resolução, frame rate e formato dos frames

Você pede resolução e frame rate no `StreamConfiguration`:

► **Código 2.5-01** – código completo no notebook companion (seção 2.5.3.4).

Na Tabela 23, você encontra diferentes resoluções para diferentes tipos de qualidade.

Tabela 23 – Diferentes resoluções para diferentes tipos de qualidade

VideoQuality	Resolução (px)
HIGH	720 × 1280
MEDIUM (padrão)	504 × 896
LOW	360 × 640

Fonte: autoria própria.

Os valores válidos de `frameRate` são **2, 7, 15, 24 ou 30 FPS** — não existe valor intermediário nem 60 FPS.

O parâmetro `compressVideo` define o formato dos frames entregues:

- `false` (padrão): o SDK decodifica internamente e entrega buffers de **pixels YUV** — prontos para exibir ou alimentar um modelo de visão.
- `true`: o SDK pula a decodificação e entrega buffers **HEVC comprimidos** — útil se você mesmo vai decodificar ou gravar. Nesse modo, frames com `isCodecConfig = true` carregam a configuração do codec (VPS/SPS/PPS) e precisam ser alimentados ao decoder antes dos frames de vídeo.

Cada frame chega como um `VideoFrame(buffer, width, height, presentationTimeUs, isCompressed, isCodecConfig)`.

Nota 33: as resoluções são "em pé" (largura × altura, ex.: 720 × 1280) — bem abaixo dos 12 MP do sensor. O stream é dimensionado para caber no Bluetooth, não para espelhar a capacidade da câmera.

13.5.3.5 Desempenho: a banda do Bluetooth manda

Resolução e frame rate reais são limitados pela conexão Bluetooth Classic entre celular e óculos. Para lidar com a banda limitada, o toolkit aplica um **rebaixamento automático em escada** (quality ladder): primeiro reduz a resolução em um degrau (ex.: HIGH → MEDIUM); se ainda faltar banda, reduz o frame rate (ex.: 30 → 24), **mas nunca abaixo de 15 FPS**.

Além disso, há **compressão por frame** que se adapta à banda disponível. Consequência contraintuitiva: a imagem pode parecer pior do que a resolução reportada (HIGH/MEDIUM) sugere. Pedir resolução menor, frame rate menor, ou ambos, deixa mais banda por frame e costuma resultar em **imagem visualmente melhor**, com menos perda de compressão.

Dois pontos finais de desempenho:

O `videoStream` lida com buffer cheio **descartando os frames mais antigos** para manter o streaming fluido. Se seu processamento por frame for lento (ex.: inferência pesada), você perderá frames — o que geralmente é o comportamento desejado em tempo real.

- Só pode haver **uma captura de foto em andamento por vez**: uma segunda chamada a `capturePhoto()` com outra pendente retorna `CaptureError.CaptureInProgress`.

Na prática: para o hackathon, resista à tentação de pedir HIGH + 30 FPS "porque é o máximo". Um pipeline de visão computacional como o do tópico 1.7 raramente precisa de mais que 15 FPS em MEDIUM ou LOW — e a imagem menos comprimida tende a melhorar a acurácia do modelo mais do que a resolução extra melhoraria. Menos banda também significa menos consumo de bateria nos óculos e no celular (tópico 1.5).

13.5.4 Na prática

Vamos abrir um stream da câmera e capturar uma foto. Os trechos seguem o guia oficial de integração Android e assumem o setup do 2.3 e o registro/permissões do 2.4 já feitos.

13.5.4.1 Passo 0 — Pré-condições

No Gradle, o módulo de câmera precisa estar entre as dependências (feito no 2.3):

► **Código 2.5-02** – código completo no notebook companion (seção 2.5.4.1).

Antes de streamar, confirme a permissão de câmera do wearable (fluxo completo no 2.4):

► **Código 2.5-03** – dispare o `RequestPermissionContract`, como visto no 2.4 — código completo no notebook companion (seção 2.5.4.1).

Atenção! Chamar qualquer API do toolkit antes de `Wearables.initialize(context)` resulta em `WearablesError.NOT_INITIALIZED`. A inicialização é uma vez por processo, no startup (tópico 2.3).

13.5.4.2 Passo 1 — Criar e iniciar a sessão

► **Código 2.5-04** – `AutoDeviceSelector` deixa o SDK escolher o dispositivo pelo usuário — código completo no notebook companion (seção 2.5.4.2).

13.5.4.3 Passo 2 — Adicionar e iniciar o stream de câmera

► **Código 2.5-05** – Config explícita: MEDIUM (504x896) a 24 FPS, frames YUV decodificados — código completo no notebook companion (seção 2.5.4.3).

Atenção! O erro mais comum deste tópico é esquecer o `stream.start()` depois do `addStream()`. O `addStream()` retorna sucesso, os collectors ficam registrados, nenhuma exceção acontece — e nenhum frame chega, porque a câmera nunca foi ativada. Se seu `videoStream` está "mudo", verifique primeiro se `start()` foi chamado e se `state` chegou a `STREAMING`.

13.5.4.4 Passo 3 — Exibir os frames

Com `compressVideo = false`, cada `VideoFrame` traz pixels YUV em `frame.buffer`, com `frame.width` e `frame.height`. A conversão YUV → Bitmap para exibição fica a cargo do app — veja a implementação de renderização no sample `CameraAccess` do repositório `meta-wearables-dat-android`.

► **Código 2.5-06** – `frame.buffer: ByteBuffer YUV (isCompressed = false)` – código completo no notebook companion (seção 2.5.4.4).

13.5.4.5 Passo 4 — Capturar uma foto

Com o streaming ativo, `capturePhoto()` é uma suspend function que retorna `DatResult<PhotoData, CaptureError>`:

► **Código 2.5-07** – código completo no notebook companion (seção 2.5.4.5).

Nota 34: se você for compartilhar a foto com outros apps, o guia oficial pede um `res/xml/file_paths.xml` para que o `FileProvider` exponha as imagens em cache.

13.5.4.6 Passo 5 — Encerrar

```
activeStream?.stop() // invalida o stream; para reuso, crie
outro com addStream
```

Código 2.5-08 · também no notebook companion, seção 2.5.4.6

Ao parar a sessão (tópico 2.4), todos os streams dela param em cascata — mas parar o stream explicitamente quando a UI sai de cena evita manter a câmera dos óculos ligada à toa.

Na prática: o "hello world" da câmera no hackathon é ver o stream chegando no app: rode o passo a passo acima com os óculos pareados — ou com o Mock Device Kit usando a câmera do próprio celular como fonte, sem hardware (tópico 2.7) — e observe o state transitar `STARTING` → `STARTED` → `STREAMING` enquanto o `ImageView` mostra a visão em primeira pessoa do usuário. Esse é o ponto de partida de qualquer solução de visão: a partir daí, basta trocar o `displayFrame` por um pipeline de inferência como o do tópico 1.7.

13.5.5 Resumo / cheatsheet

- `DeviceSession` dá acesso ao dispositivo; `Stream` (via `session.addStream(config)`) é a capability de câmera: vídeo, estado, erros e foto.
- `addStream()` registra; `stream.start()` liga a câmera. Sem `start()`, nenhum frame chega.
- Frames só são entregues no estado `STREAMING`; ciclo: `STOPPED` → `STARTING` → `STARTED` → `STREAMING` → `STOPPING` → `STOPPED` → `CLOSED`.
- `StreamConfiguration`: `videoQuality` `HIGH` 720×1280 / `MEDIUM` 504×896 (padrão) / `LOW` 360×640; `frameRate` ∈ {2, 7, 15, 24, 30}, padrão 24.
- `compressVideo = false` (padrão) entrega YUV decodificado; `true` entrega HEVC comprimido (frames `isCodecConfig` alimentam o decoder).
- Banda do Bluetooth Classic limita tudo: o ladder automático reduz primeiro resolução (um degrau), depois FPS (nunca abaixo de 15) — e pedir resolução/FPS menores reduz a compressão por frame, podendo **melhorar** a qualidade visual.

- `capturePhoto()` exige streaming ativo, aceita uma captura por vez (`CaptureError.CaptureInProgress`) e retorna `PhotoData.HEIC` ou `PhotoData.Bitmap`.
- Stop em cascata: sessão parou → streams param. Stream parado é invalidado; crie outro para voltar a streamar.

13.5.6 Referências

- ANDROID INTEGRATIONS — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/develop/dat/build-integration-android/>. Guia oficial passo a passo: manifest, Gradle, sessão, stream de câmera e captura de foto.
- STREAM INTERFACE — API REFERENCE ANDROID DAT 0.8, META. Disponível em: https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_camera_stream. Contratos de `videoStream`, `state`, `errorStream`, `start/stop` e `capturePhoto`.
- STREAMCONFIGURATION — API REFERENCE ANDROID DAT 0.8, META. Disponível em: https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_camera_types_streamconfiguration. Parâmetros `videoQuality`, `frameRate` e `compressVideo`, com defaults.
- META-WEARABLES-DAT-ANDROID — FACEBOOK/GITHUB. Disponível em: <https://github.com/facebook/meta-wearables-dat-android>. Setup do SDK via GitHub Packages e sample app completo com streaming e renderização de frames.

13.5.6.1 Para ir além



SESSION LIFECYCLE — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/develop/dat/lifecycle->

events/. Estados de sessão (RUNNING/PAUSED/STOPPED) e efeitos sobre os streams.



ANDROID TESTING COM MOCK DEVICE KIT — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/develop/dat/testing-mdk-android/>. Como testar o stream de câmera sem óculos físicos (aprofundado no tópico 2.7).

13.6 Microfone e alto-falantes via Bluetooth

13.6.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar por que o microfone e os alto-falantes dos óculos são acessados pelos perfis de áudio Bluetooth do Android, e não pela API de câmera do Device Access Toolkit (DAT).
- Comparar os perfis A2DP e HFP e escolher o perfil adequado para cada cenário (só reprodução vs. voz bidirecional).
- Implementar o roteamento de áudio para os óculos usando `AudioManager` e `setCommunicationDevice()`.
- Implementar a captura da voz do usuário com `AudioRecord` e a reprodução da resposta pelos alto-falantes open-ear.
- Antecipar as limitações de qualidade do áudio HFP (8 kHz, mono, beamforming) no design da sua solução de hackathon.

13.6.2 Pré-requisitos

- **Tópico 1.4 (Permissões em runtime)** — você vai precisar solicitar `RECORD_AUDIO` (e `BLUETOOTH_CONNECT`, já usada pelo DAT) em runtime.
- **Tópico 1.8 (STT e TTS)** — o áudio capturado aqui é a entrada do seu pipeline de voz; a saída do TTS é o que você vai reproduzir nos óculos.

- **Tópicos 2.3 e 2.4** — app já configurado com o toolkit, registrado no app Meta AI e com os óculos pareados.
- **Ferramentas:** Android Studio com o projeto do tópico 2.3; óculos Ray-Ban Meta pareados via app Meta AI. As APIs de roteamento usadas aqui (`availableCommunicationDevices` / `setCommunicationDevice`) exigem Android 12 (API 31) ou superior no smartphone.

13.6.3 Conceito / fundamentação

13.6.3.1 Duas vias de acesso: câmera pelo toolkit, áudio pelo Bluetooth

O DAT, na versão atual do developer preview, expõe as capacidades de **câmera** dos óculos: streaming de vídeo e captura de foto, como visto no 2.5. O **áudio é outra via**: microfone e alto-falantes dos óculos se comportam como qualquer headset Bluetooth e são acessados pelas APIs de áudio padrão do Android (`AudioManager`, `AudioRecord`, `AudioTrack`) por meio dos perfis de áudio Bluetooth.

Atenção! Não procure um `"session.audioStream"` no toolkit — ele não existe. A regra é: **câmera = DAT; áudio = perfis Bluetooth do Android**. Esse é o erro conceitual mais comum ao começar com o SDK: tentar obter o áudio do microfone pela sessão do DAT e concluir, errado, que "os óculos não têm API de áudio". Têm — mas é a API de áudio do próprio Android.

A boa notícia dessa separação: todo o conhecimento de áudio Android que já existe (documentação, exemplos, bibliotecas de STT/TTS do 1.8) funciona sem adaptação, porque para o Android os óculos são "só" um headset Bluetooth.

13.6.3.2 Os dois perfis de áudio Bluetooth: A2DP e HFP

Segundo a documentação oficial, o áudio do dispositivo usa dois perfis Bluetooth (Tabela 24):

Tabela 24 – Perfis de áudio utilizados pelo dispositivo

Perfil	Nome completo	Direção	Qualidade	Uso típico
A2DP	Advanced Audio Distribution Profile	Somente saída (telefone → óculos)	Alta (mídia)	Música, podcasts, áudio de mídia

HFP	Hands-Free Profile	Bidirecional (voz nos dois sentidos)	Voz, 8 kHz mono	Chamadas, assistentes de voz
------------	--------------------	--------------------------------------	-----------------	------------------------------

Fonte: autoria própria.

A escolha é ditada pela direção do fluxo: se você só precisa **reproduzir** áudio de mídia com boa qualidade, o A2DP resolve. Se precisa **capturar a voz do usuário** (com ou sem resposta de volta), o caminho é o HFP — é ele que abre o canal do microfone.

Uma analogia: o A2DP é uma caixa de som de alta fidelidade — o som só vai numa direção, mas vai bem. O HFP é um walkie-talkie — fala e escuta ao mesmo tempo, mas com qualidade de voz de telefone, não de estúdio.

13.6.3.3 O que esperar da qualidade: 8 kHz, mono e beamforming

Dois fatos da documentação oficial moldam o design da sua solução:

1. **HFP transmite áudio a 8 kHz em mono.** É qualidade de "ligação telefônica". Para fala isso costuma bastar (motores de STT são treinados com voz telefônica), mas não espere capturar música ou sons ricos do ambiente com fidelidade.
2. **O array de microfones usa beamforming para isolar e clarear a voz de quem está usando os óculos**, reduzindo significativamente o volume de sons ambientes e de outras pessoas falando. A documentação descreve isso como uma limitação esperada — mas, para o caso de uso de assistente de voz, é uma vantagem: menos ruído chegando ao seu STT.

Nota 35: beamforming é a técnica de combinar os sinais dos 5 microfones para "apontar" a escuta na direção da boca do usuário, como um zoom acústico.

Na prática: isso define o que é viável no hackathon. Ideias do tipo "assistente que responde ao que eu falo" funcionam bem — a voz do usuário chega limpa ao pipeline STT → modelo → TTS. Ideias do tipo "transcrever a conversa das pessoas ao meu redor" ou "identificar a música tocando no ambiente" tendem a falhar, porque o beamforming atenua exatamente esses sons e os 8 kHz cortam o espectro que apps como identificadores de música precisam.

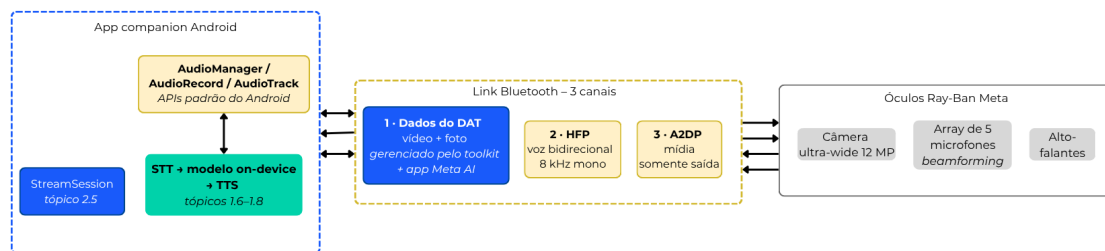
13.6.3.4 Coexistência: áudio HFP e streaming de vídeo do DAT

A documentação oficial destaca dois pontos sobre usar áudio e câmera ao mesmo tempo (Figura 24):

- As sessões do DAT **compartilham o acesso a microfones e alto-falantes com a pilha Bluetooth do sistema** nos óculos — o toolkit não "toma posse" do áudio.
- Ao planejar usar HFP e streaming simultaneamente, **configure o HFP completamente antes de iniciar a sessão de streaming** que depende de áudio. Inverter a ordem é fonte de comportamento intermitente.

Lembre também que o streaming de vídeo do 2.5 já disputa a banda do Bluetooth; somar o canal de voz HFP é mais um consumidor do mesmo link.

Figura 24 – Duas vias de acesso: câmera pelo DAT, áudio pelos perfis Bluetooth®



Fonte: autoria própria.

13.6.4 Na prática

Vamos montar o ciclo completo de voz: rotear o áudio para os óculos → capturar a fala → (processar com STT/TTS do 1.8) → reproduzir a resposta. O código abaixo segue o exemplo oficial da documentação do DAT, adaptado e comentado.

Nota 36: todos os blocos desta seção dependem de um smartphone físico com os óculos (ou um headset Bluetooth com HFP) pareados — o emulador e o Mock Device Kit não têm Bluetooth real.

13.6.4.1 Passo 0 — Permissões

Como visto no 1.4, solicite em runtime:

- `RECORD_AUDIO` — obrigatória para qualquer captura com `AudioRecord`, mesmo com microfone externo.
- `BLUETOOTH_CONNECT` — já deve estar no seu manifest desde o setup do toolkit no 2.3.

13.6.4.2 Passo 1 — Rotear o áudio para os óculos

Os óculos aparecem para o Android como um dispositivo de comunicação Bluetooth do tipo SCO (o transporte do HFP). O exemplo oficial da documentação faz o roteamento assim:

► **Código 2.6-01** – Requer API 31+ (`availableCommunicationDevices / setCommunicationDevice`) — código completo no notebook companion (seção 2.6.4.2).

Verificar: o exemplo oficial do DAT usa `AudioManager.MODE_NORMAL`; a documentação do Android Developers costuma associar `setCommunicationDevice()` a `MODE_IN_COMMUNICATION` em casos de uso de voz. Se a captura não iniciar no seu dispositivo, teste com `MODE_IN_COMMUNICATION` e confira a doc oficial das duas fontes.

Atenção! O comando `setCommunicationDevice()` pode retornar `false` (roteamento recusado) e `availableCommunicationDevices` pode simplesmente não listar os óculos — trate os dois casos com mensagem clara ao usuário ("verifique se os óculos estão conectados no app Meta AI") em vez de falhar silenciosamente.

13.6.4.3 Passo 2 — Capturar a voz do array de microfones

Com o roteamento ativo, `AudioRecord` captura normalmente — o Android entrega o áudio vindo do microfone dos óculos:

► **Código 2.6-02** – código completo no notebook companion (seção 2.6.4.3).

E o loop de leitura como um `Flow` (padrão do 1.2), pronto para alimentar o STT do 1.8:

► **Código 2.6-03** – código completo no notebook companion (seção 2.6.4.3).

Nota 37: gravar a 16 kHz é o padrão dos motores de STT, mas o sinal que chega via HFP nasce a 8 kHz — subir a taxa de amostragem não recupera qualidade que o link não transmitiu.

13.6.4.4 Passo 3 — Reproduzir a resposta nos alto-falantes open-ear

Se você usa o `TextToSpeech` do 1.8, a fala sintetizada já sai pelo dispositivo de áudio ativo — os óculos. Para reproduzir PCM bruto (por exemplo, de um TTS on-device que gera samples), use `AudioTrack` marcando o áudio como comunicação de voz, para que ele siga o mesmo roteamento do Passo 1:

► **Código 2.6-04** – código completo no notebook companion (seção 2.6.4.4).

13.6.4.5 Passo 4 — Liberar o roteamento

Quando a interação de voz terminar, devolva o áudio ao padrão do sistema — senão todo áudio do telefone continua preso no canal de voz de 8 kHz:

► **Código 2.6-05** – código completo no notebook companion (seção 2.6.4.5).

13.6.4.6 Erro comum: iniciar o streaming de vídeo antes de configurar o HFP

Se sua solução usa câmera e voz (o caso típico no hackathon), a documentação oficial é explícita: configure o HFP por completo **antes** de iniciar a `StreamSession` que depende de áudio. O sintoma de inverter a ordem é captura de voz que "às vezes funciona": o canal SCO não sobe de forma confiável com o streaming já ativo. A correção é sequenciar — rotear o áudio, confirmar que o dispositivo de comunicação está ativo e só então chamar o `session.addStream(...)` do 2.5.

Na prática: para o hackathon, encapsule os Passos 1–4 numa classe única (ex.: `GlassesAudioManager`) com `iniciar()/liberar()`, e teste o

pipeline de voz com um fone Bluetooth comum com HFP antes de ter os óculos em mãos — para o Android, ambos são o mesmo tipo de dispositivo, e você libera os óculos da equipe para quem está testando a câmera. A simulação de dispositivo do toolkit será vista no 2.7.

13.6.5 Resumo / cheatsheet

- Câmera = DAT (tópico 2.5); microfone e alto-falantes = perfis de áudio Bluetooth padrão do Android. Não existe API de áudio no toolkit.
- A2DP: somente saída, alta qualidade (mídia). HFP: bidirecional, voz — é o perfil que abre o microfone.
- HFP entrega 8 kHz mono; beamforming do array de 5 microfones isola a voz do usuário e atenua o ambiente.
- Roteamento: `audioManager.availableCommunicationDevices` → achar `TYPE_BLUETOOTH_SCO` → `setCommunicationDevice()` (API 31+).
- Captura: `AudioRecord` com fonte `VOICE_COMMUNICATION`, mono, PCM 16-bit, em `Dispatchers.IO`.
- Reprodução: TTS sai direto pelos óculos; PCM bruto via `AudioTrack` com `USAGE_VOICE_COMMUNICATION`.
- Usando voz + vídeo juntos: configure o HFP **antes** de iniciar a sessão de streaming.
- Sempre chame `clearCommunicationDevice()` ao terminar; permissões necessárias: `RECORD_AUDIO` + `BLUETOOTH_CONNECT`.

13.6.6 Referências

- USE DEVICE MICROPHONES AND SPEAKERS — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/microphones-and-speakers/>.

Página oficial deste tópico: perfis A2DP/HFP, nota de qualidade (8 kHz/beamforming) e o código de roteamento Android usado aqui.

- AUDIOMANAGER — ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/media/AudioManager>. Referência de `availableCommunicationDevices`, `setCommunicationDevice()`, `clearCommunicationDevice()` e modos de áudio.
- AUDIORECORD — ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/media/AudioRecord>. Referência da API de captura PCM usada no Passo 2.
- META WEARABLES DEVICE ACCESS TOOLKIT FOR ANDROID — GITHUB (FACEBOOK). Disponível em: <https://github.com/facebook/meta-wearables-dat-android>. Repositório oficial com sample app para comparar sua integração.

13.6.6.1 Para ir além



AUDIODEVICEINFO — ANDROID DEVELOPERS. Disponível em: <https://developer.android.com/reference/android/media/AudioDeviceInfo>. Todos os tipos de dispositivo de áudio (SCO, A2DP, BLE) e como inspecioná-los.



SESSION LIFECYCLE — META WEARABLES DEVELOPER CENTER. Disponível em: <https://wearables.developer.meta.com/docs/lifecycle-events/>. Como o ciclo de vida da sessão do DAT interage com o restante do app (revisto no 2.4).

13.7 Desenvolvimento sem *hardware* (*Mock Device Kit*)

13.7.1 Objetivos de aprendizagem

Ao final, você será capaz de:

- Explicar o que é o Mock Device Kit e como ele se encaixa na pilha do Device Access Toolkit (DAT) sem exigir mudanças no código do app.
- Configurar e parear um dispositivo simulado, controlando seu estado (ligado, vestido, aberto/fechado).
- Simular permissões do toolkit e streaming de mídia usando arquivo de vídeo, câmera do celular ou imagem de captura.
- Organizar o fluxo de trabalho da fase online do hackathon para desenvolver e testar a solução inteira antes de receber os óculos.

13.7.2 Pré-requisitos

- Tópicos 2.3 (Setup do projeto com o DAT), 2.4 (Registro e sessão) e 2.5 (Câmera e streaming) — o Mock Device Kit simula exatamente esses fluxos.
- Tópico 1.4 (Permissões em runtime) — a diferença entre permissões do Android e permissões do toolkit reaparece aqui.
- Android Studio Flamingo ou mais recente e um celular Android 10+ para rodar o app companion. [Verificar](#): requisitos de versão na página de Setup oficial, pois o toolkit está em developer preview.

13.7.3 Conceito / fundamentação

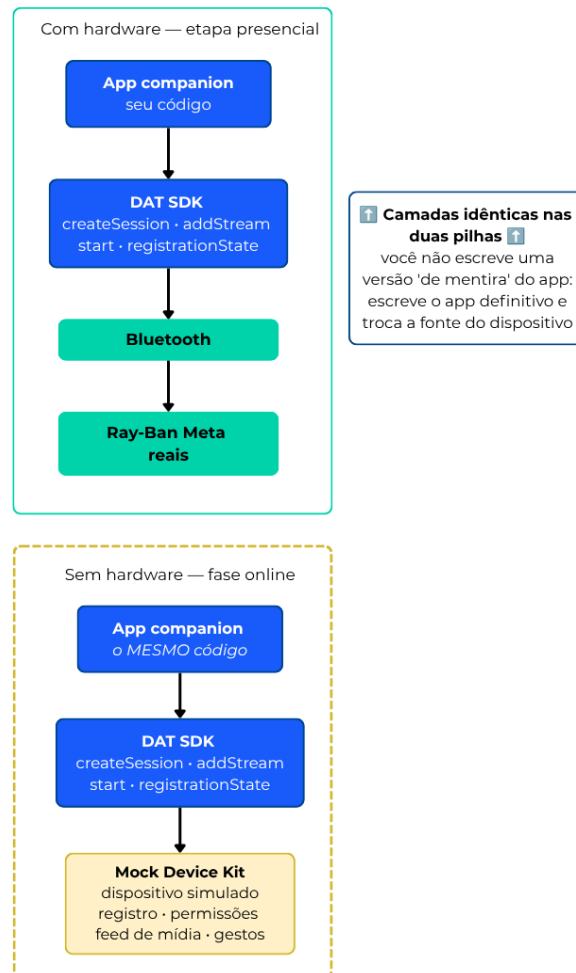
13.7.3.1 O que é o Mock Device Kit

O Mock Device Kit (MDK) é um componente do Device Access Toolkit que permite construir e testar integrações com os óculos Meta **sem o hardware real**. Ele fornece um dispositivo simulado que espelha as capacidades e o comportamento dos óculos: streaming de câmera, captura de foto, permissões e mudanças de estado do dispositivo.

A analogia útil é a de um dublê de cinema: para a câmera (o seu código), o dublê se comporta como o ator principal. Quando o MDK está habilitado, ele simula a pilha inteira do SDK — inclusive a conexão do app (o registro visto no 2.4) e as solicitações de permissão (Figura 25). Seu código funciona do mesmo jeito falando

com um dispositivo real ou com um mock: você não escreve uma versão "de mentira" do app; escreve o app definitivo e apenas troca a fonte do dispositivo.

Figura 25 – A mesma pilha, duas bases: hardware real e Mock Device Kit



Fonte: autoria própria.

13.7.3.2 Por que isso é crítico no cronograma do AI Glasses Brasil

Os óculos Ray-Ban Meta só chegam às equipes na etapa presencial. Ou seja: **toda a fase online do programa acontece sem o hardware**. Sem o MDK, as equipes só poderiam começar a integração real no presencial — tarde demais para um hackathon.

Com o MDK, a fase online deixa de ser "estudo teórico" e vira desenvolvimento de verdade: você implementa registro, sessão, streaming, os modelos de IA on-device (Módulo 1) e a UI, tudo validado contra o dispositivo

simulado. No presencial, o trabalho que sobra é trocar o mock pelos óculos reais e ajustar o que só o hardware revela (latência real do Bluetooth, iluminação da câmera, consumo de bateria).

Na prática: planeje a divisão do time assumindo que a solução inteira será desenvolvida sem hardware: quem cuida do pipeline de visão (1.7) usa um vídeo H.265 como feed simulado; quem cuida de voz (1.8) desenvolve com o microfone do próprio celular; quem cuida da integração DAT valida sessão e permissões contra o mock. No dia do presencial, o "Start streaming" que funcionava com o mock deve funcionar com os óculos sem mudar uma linha da lógica do app.

13.7.3.3 O que o MDK simula (e o que não simula)

O dispositivo simulado cobre quatro áreas:

- **Estado do dispositivo** — ligar/desligar (`powerOn/powerOff`), vestir/tirar (`don/doff`) e abrir/fechar as hastes (`unfold/fold`). Esses eventos são os mesmos que o SDK observa num dispositivo real (wear detection, hinge events).
- **Registro e permissões** — ao habilitar o MDK, o estado de registro do app já transita para `Registered` por padrão, e você pode configurar respostas de permissão (concedida, negada) para testar os dois caminhos.
- **Mídia** — o feed da câmera dos óculos é substituído por um arquivo de vídeo H.265 ou pela câmera do próprio celular; a captura de foto retorna uma imagem que você define.
- **Gestos de toque** — simulação de tap e tap-and-hold na haste: durante um stream ativo, tap alterna pausa/retomada e tap-and-hold encerra a sessão.

O que o mock não substitui: características físicas do hardware real — qualidade óptica da câmera ultra-wide, comportamento do array de microfones com beamforming, autonomia de bateria e a instabilidade real do link Bluetooth.

E há uma ausência que **não é lacuna, e sim consequência da arquitetura:** o MDK não simula captura de áudio. Faz todo sentido — como visto no 2.6, o

microfone e os alto-falantes dos óculos não passam pelo DAT, e sim pelos perfis de áudio Bluetooth do Android. O MDK simula a pilha do DAT; áudio nunca esteve nela, então não há o que simular. A documentação oficial lista exatamente o que o dispositivo simulado espelha — streaming de câmera, captura de foto, permissões, estado do dispositivo e gestos de toque — e áudio não aparece.

Consequência prática para a fase online: desenvolva o pipeline de voz com **um fone Bluetooth comum com suporte a HFP**, como recomendado no 2.6.

13.7.3.4 Duas formas de usar

Há dois caminhos complementares:

1. **UI de debug do sample CameraAccess** — o app de exemplo oficial traz uma folha de controles do MDK (disponível apenas em *debug builds*, as compilações de desenvolvimento). É o jeito mais rápido de ver o mock funcionando sem escrever código.
2. **API do MDK no seu próprio app/testes** — a classe `MockDeviceKit` é o ponto de entrada programático: você habilita o mock, pareia dispositivos e configura mídia via código, inclusive em testes instrumentados.

13.7.4 Na prática

13.7.4.1 Caminho 1 — Explorar o MDK pela UI de debug do sample

Pré-condição: o repositório oficial `meta-wearables-dat-android` clonado e o sample CameraAccess compilando (setup do 2.3).

1. **Habilite o MDK:** rode o sample em modo debug, toque no ícone de debug (joaninha) na tela inicial e toque em **Enable MockDeviceKit**. Isso ativa o sistema de mock, que simula conexão e permissões — o sample completa o registro automaticamente e o app passa da etapa de conexão sem óculos e sem o app Meta AI.
2. **Pareie um dispositivo simulado:** no menu de debug, toque em **Pair RayBan Meta**. Um card do dispositivo mock aparece na folha. É possível parear até 3 dispositivos simultâneos; **Unpair** remove.

3. **Mude o estado do dispositivo** pelos toggles do card: **Power** (liga), **Don** (veste — e já abre as hastes automaticamente), **Unfold** (abre as hastes). O dispositivo precisa estar ligado e vestido para o streaming começar.
4. **Configure a fonte de mídia** expandindo o card: no *camera source picker*, escolha **Front Camera/Back Camera** (feed ao vivo da câmera do celular) ou **Video File** (vídeo da galeria). Para captura de foto, **Select image** define a imagem retornada — com câmera do celular como feed, a captura tira um still ao vivo e dispensa essa configuração.
5. **Inicie o streaming**: feche a folha de debug e toque em **Start streaming**. O feed configurado aparece como se viesse dos óculos.
6. **Simule gestos**: use os botões **Tap** (pausa/retoma o stream) e **Tap and Hold** (encerra a sessão) no card do dispositivo.

Atenção! O MDK exige vídeos no formato H.265 (HEVC). O sample Android **não** converte automaticamente. Se o stream não inicia ou fica preto com um vídeo da galeria, transcode antes com FFmpeg — a doc oficial sugere, por exemplo:

```
ffmpeg -i entrada.mp4 -c:v libx265 -tag:v hvc1 -vf "scale=540:960" saida.mov
```

Código 2.7-01 · também no notebook companion, seção 2.7.4.1

13.7.4.2 Caminho 2 — Usar a API do MDK no seu app

O fluxo programático segue a mesma lógica da UI. Código baseado nos exemplos da API reference (Android DAT **0.8.0**; confira a versão atual na doc).

► **Código 2.7-02** – código completo no notebook companion (seção 2.7.4.2).

Pontos que merecem destaque:

- `enable()` aceita um `MockDeviceKitConfig`; com `initiallyRegistered = false` o app começa **não registrado**, o que permite testar a UI do fluxo de registro do 2.4.

- Para testar o caminho de permissão negada sem mexer no app:

► **Código 2.7-03** – `checkPermissionStatus(CAMERA)` passará a responder Denied — código completo no notebook companion (seção 2.7.4.2).

- Em testes instrumentados, o padrão oficial é um caso de teste base que obtém o `MockDeviceKit` no `@Before`, concede as permissões Android via shell (`pm grant ... BLUETOOTH_CONNECT e CAMERA`) e chama `mockDeviceKit.disable()` no `@After` (e `enable()` no `@Before`) para limpar o estado entre testes. O guia de testes usa `device.services.camera` (não há `getCameraKit` no 0.8).

Na prática: estruture seu app para que o MDK fique atrás de uma flag de debug (por exemplo, habilitar o mock apenas quando `BuildConfig.DEBUG` for true, como faz o sample). Assim, o mesmo APK que a equipe usa a fase online inteira vira o APK do presencial: desligou o mock, a sessão real (`createSession` → `start` → `addStream`) passa a receber os óculos reais e todo o pipeline de IA construído no Módulo 1 continua funcionando sem alteração.

13.7.4.3 Erros comuns

- **"Start streaming" não habilita / dispositivo não aparece** → o mock está pareado, mas não está ligado e vestido. Chame `powerOn()` e `don()` (ou ligue os toggles Power e Don na UI de debug).
- **Stream não inicia com arquivo de vídeo** → o vídeo não está em H.265. Transcode com FFmpeg; prefira arquivos pequenos, pois vídeos grandes elevam o uso de memória.
- **Feed da câmera do celular falha** → a permissão Android `CAMERA` não foi concedida em runtime (isso é permissão do *Android*, não do toolkit — a distinção do 1.4).
- **Foto capturada aparece "deitada"** → comportamento documentado: a imagem retornada pelo mock é rotacionada 90° para reproduzir o dispositivo real. Trate a rotação no seu pipeline, não como bug.

13.7.5 Resumo / cheatsheet

- Mock Device Kit = dispositivo simulado que espelha os óculos Meta: estado, registro, permissões, streaming, captura e gestos de toque — sem hardware.
- O código do app é o mesmo com mock ou dispositivo real; o MDK troca a base da pilha do SDK, não a sua lógica.
- Crítico no AI Glasses Brasi: os óculos só chegam no presencial — toda a fase online roda sobre o MDK.
- Fluxo mínimo: `getInstance(context)` → `enable()` → `pairGlasses(GlassesModel.RAYBAN_META)` → `powerOn()` + `don()` → configurar mídia → sessão normal do SDK.
- Streaming só inicia com o dispositivo **ligado e vestido**; até 3 mocks pareados ao mesmo tempo.
- Mídia simulada: vídeo em H.265 (obrigatório), câmera do celular (frontal/traseira, exige permissão Android `CAMERA`) ou imagem fixa para captura (retorna rotacionada 90°).
- Permissões do toolkit: `permissions.set(...)` controla o status consultado; `setRequestResult(...)` controla a resposta da solicitação (padrão: `Granted`).
- `disable()` restaura a pilha real e limpa o estado — use `enable()/disable()` entre testes (não há `reset()` no 0.8).

13.7.6 Referências

- MOCK DEVICE KIT — META WEARABLES DEVELOPER CENTER.
Disponível em:
<https://wearables.developer.meta.com/docs/develop/dat/mock-device-kit/>.
Visão geral do kit e passo a passo da UI de debug do sample
CameraAccess (parear, estados, mídia, gestos).

- HOW TO TEST WITH MOCK DEVICE KIT ON ANDROID — META WEARABLES DEVELOPER CENTER. Disponível em:
<https://wearables.developer.meta.com/docs/develop/dat/testing-mdk-android/>. Padrões para testes instrumentados: caso de teste base, permissões via shell, feed e captura mockados.
- MOCKDEVICEKIT CLASS (ANDROID DAT API REFERENCE) — META WEARABLES DEVELOPER CENTER. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_mockdevice_mockdevicekit. API do ponto de entrada: `enable/disable`, `pairGlasses`, `permissions`, `pairedDevices`.
- META-WEARABLES-DAT-ANDROID (REPOSITÓRIO OFICIAL) — META/GITHUB. Disponível em:
<https://github.com/facebook/meta-wearables-dat-android>. Código do sample CameraAccess com a UI de debug do MDK.

13.7.6.1 Para ir além



MOCKCAMERAKIT INTERFACE — META WEARABLES DEVELOPER CENTER. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_mockdevice_api_camera_mockcamerakit. Detalhes de `setCameraFeed` (arquivo × câmera do celular) e `setCapturedImage`.



MOCKRAYBANMETA INTERFACE — META WEARABLES DEVELOPER CENTER. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_mockdevice_api_mockraybanmeta. Todos os métodos de estado do dispositivo simulado (`don/doff`, `fold/unfold`, `powerOn/powerOff`).



MOCKPERMISSIONS INTERFACE — META WEARABLES DEVELOPER CENTER. Disponível em:
https://wearables.developer.meta.com/docs/reference/android/dat/0.6/com_meta_wearable_dat_mockdevice_api_permissions_mockpermissions. Controle fino das respostas de permissão do toolkit.



 **CEIA** |  **Meta**